# PK Reference backend

A study guide to the Kafka, Spring Boot and PostgreSQL service: every file, every method, and how they fit together.

Built on 12 October 2026 by backend/docs/study-guide/build_study_guide.py.
The file-by-file part is read from the comments in the source, so it matches what the files say.

# Contents


**Part 1** **The system in ten minutes**
What it does, the stack, the architecture diagram, and where each file sits.

**Part 2** **Follow the data**
An event, a ladder month and an API request, step by step with class and method names. What happens on failure.

**Part 3** **Spring, as this project uses it**
Beans, @Bean and @Component, settings, timers, Kafka listeners, the web and database layers.

**Part 4** **Kafka, as this project uses it**
Topics, keys, groups, compaction, delivery guarantees, the settings and why.

**Part 5** **The database**
Every table, Flyway, JdbcTemplate against JPA, transactions and the version counter.

**Part 6** **Signing in**
API keys, access tokens, what a JWT holds and how it is verified.

**Part 7** **The API**
Every endpoint and the method behind it.

**Part 8** **File by file**
Every class, annotation and method, read from the code's own comments.

**Part 9** **The tests**
The five kinds of test, golden files, and every test class.

**Part 10** **Running it**
Commands, settings, the server set-up, rebuilding from the topics.

**Part 11** **Design decisions, and questions to expect**
Answers to say aloud, including the system's limits.

# Part 1. The system in ten minutes

## What it is

The PK Reference backend is a small data pipeline and read-only API for a Pokémon reference app. It does three things:

1. **Collects.** It polls two public sources: **Limitless**, for tournament events (who played, their teams, who beat whom), and **Smogon**, for monthly statistics of the Pokémon Showdown ladder.
2. **Keeps.** Everything fetched goes through **Kafka** topics that are never emptied, and from there into **PostgreSQL** tables. The topics are the record of what was fetched; the tables are a view of it that can be rebuilt by reading the topics again.
3. **Answers.** A JSON API at `/v1` serves insights worked out from the tables: how often each Pokémon is used, how it does in top cuts, win rates from match results, trends, common sets, cores and archetypes, and the ladder's numbers.

> **Why it exists.** Before it, every copy of the app crawled Limitless by itself, which is slow, repeats the same work on every device and runs into Limitless's rate limit (50 requests in 5 minutes). One server crawling once for everyone is lighter on the source, and it can keep match results, which lets it compute things a device can't, such as win rates that leave out mirror matches.

## The stack

| Piece | Version | What it does here |
|---|---|---|
| Java | 21 | Records, text blocks, `var`, switch expressions are used throughout. |
| Spring Boot | 3.4.5 | Wires the application together, runs the web server (embedded Tomcat), the scheduler and the Kafka listeners. |
| Apache Kafka | 3.9 broker, one node in KRaft mode | The log between "fetched" and "stored". Five topics, three of them kept for good. |
| PostgreSQL | 17 | The tables the API reads. Twelve of them. |
| Flyway | | Creates and changes the tables from numbered SQL files. |
| springdoc | 2.8.6 | Swagger UI and the OpenAPI document, generated from the controllers' annotations. |
| Nimbus JOSE + JWT | 10.10 | Signs and verifies access tokens. |
| JUnit 5, Mockito, Testcontainers, embedded Kafka | | 137 tests in 30 classes. |
| Docker Compose | | Runs Kafka and Postgres locally; on a server, the API too. |

## Size

51 Java source files (5,581 lines with comments) in eleven packages, five SQL migrations, and 30 test classes. Small enough to know completely, which is the aim of this guide.

## The one idea to hold on to

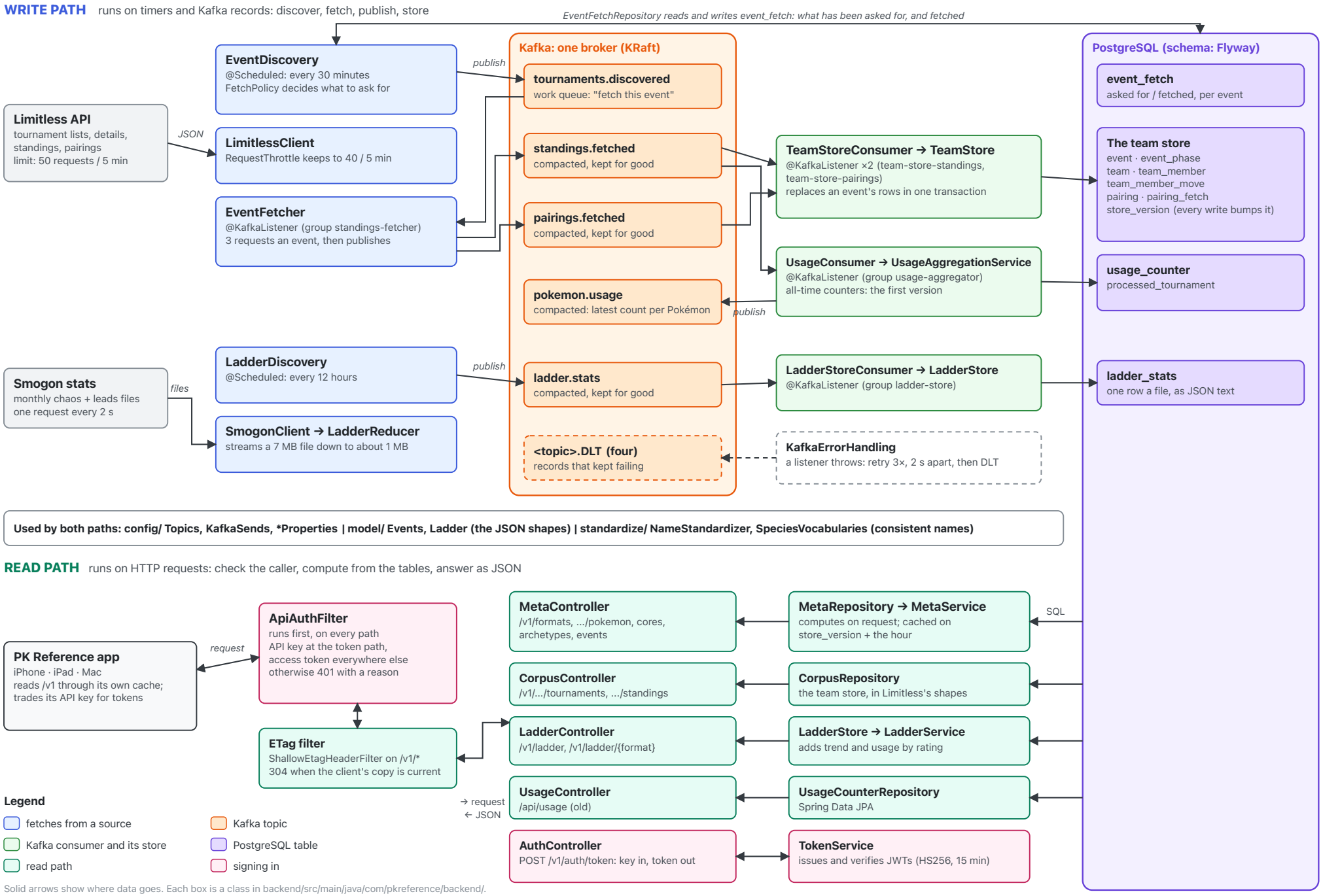
The system is split at Kafka into a **write path** and a **read path**.

- The **write path** runs on timers and on Kafka records, never on a user's request: discover what to fetch, fetch it, publish it, store it. It is slow on purpose (rate limits) and built to be repeated safely.
- The **read path** runs on HTTP requests: a filter checks the caller, a controller calls a service, the service reads the tables and computes the answer, and the answer is cached. It never calls Limitless, Smogon or Kafka.

The two paths share only the database. If the sources or Kafka are down, the API keeps answering from what is stored.

**Next page:** the architecture diagram. Every box is a class; the arrows show where data goes.

# The architecture: where each class sits, and what flows between them




Solid arrows show where data goes. Each box is a class in backend/src/main/java/com/pkreference/backend/.

## Where each file sits

The source tree, with one line for each class. The packages are listed in the order data flows through them, not alphabetically.

`backend/src/main/java/com/pkreference/backend/`

| | |
|---|---|
| **(the root package)** | |
| BackendApplication | The program's entry point, and the root of everything Spring builds. |
| **config/** | |
| KafkaErrorHandling | What happens when a @KafkaListener method throws. |
| KafkaSends | Waits for a Kafka send, so a failure (a record over the topic's size limit, the broker down) throws where it happened. |
| LimitlessProperties | Limitless and how events are found. |
| SmogonProperties | Smogon's monthly ladder stats and how they're fetched: the settings under pkref.smogon in application.yml, bound into this record by Spring as LimitlessProperties is. |
| Topics | The Kafka topics, declared in code. |
| **model/** | |
| Events | Wire types: the shapes of the JSON that travels between Limitless, the Kafka topics and the app. |
| Ladder | Ladder stats, reduced from one of Smogon's monthly files to what the app shows (BackendIntegration-PHASE5.md §3). |
| **ingest/** | |
| EventDiscovery | Walks each format's tournament list back through the lookback and asks for the events to fetch (FetchPolicy) on tournaments.discovered. |
| EventFetchRepository | The event_fetch table: what discovery has asked for, and when the fetcher got it. |
| EventFetcher | tournaments.discovered -> the event's details, standings and pairings -> standings.fetched and pairings.fetched. |
| FetchPolicy | Which listed events to fetch: the app's rules (TeamCorpusStore). |
| LimitlessClient | Thin client for the Limitless play API, same endpoints as the app's LimitlessAPI actor. |
| RequestThrottle | Keeps requests under a limit of `limit` in each `window`, as Limitless's keyless API allows (50 in 5 minutes): a sliding window of the requests sent, plus what Limitless says is left. |
| **store/** | |
| PairingResult | How a Limitless match ended. |
| TeamStore | Every event's teams and matches, as fetched: the tables Phase 2's insights read. |
| TeamStoreConsumer | standings.fetched and pairings.fetched -> the team store. |
| **standardize/** | |
| NameDictionary | Canonical spellings for one kind of name, with exact (normalized) and typo-tolerant lookup. |
| NameStandardizer | Canonicalizes the free-text names players type into decklists (items, abilities, moves, natures, Tera types), so "MIRACLE SEED", "Miracle seed" and "miracle seed" count as one. |
| SpeciesNames | The app's name helpers, ported rule for rule so species keys match it: toID (ShowdownCalc.swift), LimitlessSpeciesResolver.words (LimitlessTeamImport.swift) and TeamQueryParser.tokens. |
| SpeciesVocabularies | Each bundled regulation's SpeciesVocabulary, by Limitless format ("M-C" is champions-m-c.json). |
| SpeciesVocabulary | One regulation's species, as the app's TeamSearchVocabulary knows them: what species a Limitless member is (with the form words the regulation lists) and which Mega its held stone makes. |
| **insight/** | |
| MetaController | API v1: read-only, the same for everyone, and cached. |
| MetaRepository | Reads the team store for the insights. |
| MetaResponses | The /v1 responses. |
| MetaService | The insights, worked out from the team store when asked: usage in a window, its share of top-cut teams, its trend, and each Pokémon's sets, items, moves and teammates. |
| MetaWebConfig | ETags for /v1, worked out from each response's body: a matching If-None-Match gets a 304. |
| MetaWindow | The time a response covers: the last 14 or 30 days, or the whole regulation (every stored event). |
| **corpus/** | |
| CorpusController | The stored events in Limitless's shapes, for the app's Team Search corpus: the same two calls it makes to Limitless (a page of a format's tournaments, and one tournament's standings),... |
| CorpusRepository | The team store read back in Limitless's own shapes (Events.Tournament and Events.Standing), so the app's Team Search corpus can be built from the server with the code that builds it from... |

| | |
|---|---|
| **ladder/** | |
| LadderController | API v1's ladder: read-only and cached as the rest of /v1 is (MetaController). |
| LadderDiscovery | Looks through Smogon's newest months for Champions ladder stats not stored yet, and fetches, reduces and publishes each on ladder.stats. |
| LadderFormat | One of Smogon's Champions ladder formats, read from its id: "gen9championsvgc2026regmc" is M-C doubles, with "bo3" on the end best of three; "gen9championsbssregmc" is M-C singles... |
| LadderReducer | Cuts one of Smogon's chaos files, and its leads file, down to a Ladder.Stats (BackendIntegration-PHASE5.md §3). |
| LadderResponses | The /v1/ladder responses. |
| LadderService | The ladder stats, as /v1/ladder serves them: a stored record, with what the other stored files add to it (each Pokémon's change since the month before, and its usage at the other ratings). |
| LadderStore | The ladder_stats table: each reduced month of Smogon's stats, as ladder.stats delivered it. |
| LadderStoreConsumer | ladder.stats -> the ladder_stats table. |
| SmogonClient | Smogon's monthly stats (https://www.smogon.com/stats/): plain directory listings and files, the ones the app's SmogonLiveFetcher reads. |
| **security/** | |
| ApiAuthFilter | Who gets in, checked before anything else sees the request. |
| ApiAuthProperties | Who may use the API, and how (README, "Signing in to the API"). |
| AuthController | Trades an API key for an access token. |
| SecurityConfig | Signing in to the API, put together: the tokens, and the check ahead of every other filter and on every path but Swagger UI's (/v1 and the older /api/usage). |
| TokenService | The API's access tokens: JSON Web Tokens (JWTs), which an API key is traded for and every other request then carries. |
| **usage/** | |
| ProcessedTournament | Marker written in the same transaction as the counters, so replays never double-count. |
| ProcessedTournamentRepository | Which events have been counted. |
| UsageAggregationService | Turns one event's standings into counter changes: how many teams ran each Pokémon, and for each Pokémon how many ran each item, ability, move, Tera type and nature. |
| UsageConsumer | standings.fetched -> counters -> pokemon.usage. |
| UsageCounter | One counter row. |
| UsageCounterRepository | The counters' queries. |
| **api/** | |
| UsageController | The first version's API: all-time usage from the counters, at /api/usage. |

# Part 2. Follow the data

Three walks through the code. Each step names the class and method that does it; Part 8 has every one of them in detail.

## 2.1 The life of a tournament event

1. **The timer fires.** Every 30 minutes Spring's scheduler calls `EventDiscovery.poll()` (`@Scheduled`). For each configured format it calls `discover(format)`.
2. **List.** `discover` asks `LimitlessClient.tournaments(format, page)` for the newest events, a page of 50 at a time, walking back until a whole page is older than the lookback (7 days). Every request first waits its turn in `RequestThrottle.acquire()`.
3. **Decide.** For each listed event, `FetchPolicy.shouldRequest(...)` answers "ask for it now?" from the event itself and from what the `event_fetch` table remembers (`EventFetchRepository.find`): at least 16 players, already started, not already final, not already waiting in the queue, and if fetched before, not within the last 6 hours.
4. **Ask.** For each "yes", discovery publishes a `TournamentDiscovered` record to the topic `tournaments.discovered`, keyed by the event's id, waits for the broker to confirm (`KafkaSends.await`), and notes it in `event_fetch` (`markRequested`).
5. **Fetch.** `EventFetcher.onDiscovered(...)` (`@KafkaListener`, group `standings-fetcher`) receives the record and makes three throttled requests: the event's details, standings and pairings. It works out whether the standings are final (`FetchPolicy.isFinal`: fetched 48 hours or more after the start).
6. **Publish.** The fetcher sends a `StandingsFetched` record to `standings.fetched` and a `PairingsFetched` record to `pairings.fetched`, both keyed by the event's id, waits for both, then calls `markFetched`. Only then does the method return and the offset move on.
7. **Store.** Two independent consumer groups read `standings.fetched`:
   - `TeamStoreConsumer.onStandings` (group `team-store-standings`) calls `TeamStore.putStandings`, which in one transaction deletes the event's old rows and inserts its event, phases, teams, members and moves. Each member gets its species key from `SpeciesVocabularies.identify`. The store's version counter goes up by one.
   - `UsageConsumer.onStandings` (group `usage-aggregator`) calls `UsageAggregationService.apply`, which adds the event to the all-time counters once it is final, and marks it processed in the same transaction.

   `TeamStoreConsumer.onPairings` (group `team-store-pairings`) stores the matches through `TeamStore.putPairings`.
8. **Serve.** The next request to `/v1/formats/M-C/pokemon` finds a new store version, so `MetaService` recomputes from the tables, and the new event is in the numbers.
9. **Again, until final.** An event fetched before its standings settled is asked for again every 6 hours (step 3) until a fetch comes 48 hours after its start. Each later fetch replaces the earlier rows.

## 2.2 The life of a ladder month

1. **The timer fires.** Every 12 hours `LadderDiscovery.poll()` calls `discover()`.
2. **List.** `SmogonClient.months()` reads Smogon's index page; for each of the two newest months, `chaosFiles(month)` reads that month's listing. The client waits 2 seconds between requests.
3. **Choose.** Each file name is split into a format and a rating. `LadderFormat.parse` recognizes Champions doubles and singles formats of a known regulation; everything else is skipped. `LadderStore.has(...)` skips files already stored by the current rule.
4. **Fetch and reduce.** For each file left, discovery gets the small leads table (`SmogonClient.leads`) and then streams the big statistics file (`SmogonClient.chaos`) straight into `LadderReducer.reduce`, which reads one Pokémon at a time and keeps only what the app shows. Up to 7 MB becomes about 1 MB.
5. **Publish.** The reduced `Ladder.Stats`, wrapped in `StatsFetched` with the time, goes to the topic `ladder.stats`, keyed `format|month|rating`.
6. **Store.** `LadderStoreConsumer.onStats` (group `ladder-store`) calls `LadderStore.put`, which keeps the record as JSON text in `ladder_stats` unless a newer one is already there.
7. **Serve.** `GET /v1/ladder/{format}` reaches `LadderService.stats`, which reads the row and adds each Pokémon's change since the previous month and its usage at the other ratings.

A published month never changes, so each file is fetched exactly once. After the first run a poll is three small listing requests and nothing else, until a new month appears.

## 2.3 The life of an API request

Take `GET /v1/formats/M-C/pokemon?window=30d` with an access token.

1. **Tomcat** accepts the connection and runs the servlet filters in order.
2. **`ApiAuthFilter`** runs first. With keys configured it reads the `Authorization: Bearer ...` header and has `TokenService.verify`

check the token's algorithm, signature, expiry and claims. A bad or missing credential ends here with a 401. With no keys configured the filter lets everything through.

3. **The ETag filter** (registered in `MetaWebConfig` for `/v1/*`) wraps the response so it can hash the body later.
4. **Spring MVC** matches the path to `MetaController.pokemon(format, window)`, converting `{format}` and `?window=` into its arguments.
5. **The controller** turns "30d" into a `MetaWindow` (anything else is a 400) and calls `MetaService.pokemon`.
6. **The service** builds a cache key from the request, the store's version and the hour. On a hit it returns the remembered answer. On a miss it reads rows through `MetaRepository`, counts in memory, and remembers the result.
7. **Jackson** writes the returned record as JSON. The controller added `Cache-Control: max-age=900`.
8. **On the way out** the ETag filter hashes the body: if the request's `If-None-Match` has the same hash, the client gets `304 Not Modified` and no body. The auth filter's wrapper has turned `public` into `private` in the caching header. Tomcat gzips bodies over 2 KB.

## 2.4 When things go wrong

| What happens | What the system does |
|---|---|
| Limitless is down or answers an error while an event is being fetched | `EventFetcher.onDiscovered` throws. The error handler retries the record 3 times, 2 seconds apart, then parks it on `tournaments.discovered.DLT`. Nothing was marked fetched, so after 24 hours discovery treats the request as lost and asks again. |
| Limitless answers 429 (too many requests) | `LimitlessClient.withRetry` waits for the time its `Retry-After` header gives (at most 30 seconds) and tries again, up to 3 attempts. The throttle also reads Limitless's own count of what is left and stops sending before the limit. |
| The broker is down when discovery publishes | `KafkaSends.await` throws after 30 seconds. Discovery logs it and does not mark the event requested, so the next poll asks again. |
| The server stops between publishing standings and committing the offset | On restart the fetcher gets the same record again and fetches the event again. The stores replace the event's rows, and the counters skip an event already marked processed, so the repeat changes nothing. |
| A record can't be stored (a bug, a constraint) | The transaction rolls back, the record is retried, then goes to its DLT. Other records carry on. |
| Kafka and both sources are down | The API keeps answering from Postgres. Nothing new arrives until they are back. |
| Postgres is down | Requests fail with a 500. A listener fails, retries 3 times over 6 seconds, and the record goes to its DLT. Nothing is lost, because `standings.fetched`, `pairings.fetched` and `ladder.stats` keep every record: once the database is back, resetting the stores' consumer groups reads them in again. (A longer back-off would ride out a short outage by itself; see Part 11.) |
| The tables are lost or a counting rule changes | Reset the consumer groups' offsets to the start and the tables are rebuilt from the topics (README, "Rebuilding the counters and the store"). |

# Part 3. Spring, as this project uses it

Spring's job here is to build the objects and connect them. This part explains each mechanism once, with where the code uses it. If an interviewer points at an annotation, the answer is in this part.

## 3.1 Beans and dependency injection

A **bean** is an object that Spring creates and keeps, normally exactly one of each (a singleton). The set of all beans is the **application context**. Classes don't create their collaborators: they declare them as constructor parameters, and Spring passes them in. That is **dependency injection**.

```
@Component
public class EventFetcher {
    public EventFetcher(LimitlessClient client, EventFetchRepository fetches,
                        KafkaTemplate<String, Object> kafka, LimitlessProperties props, Clock clock) { ... }
}
```

Spring sees one constructor, finds a bean of each parameter's type, and calls it. No `@Autowired` is needed when a class has a single constructor. (`LimitlessClient` and `SmogonClient` have two, one for tests, so the one Spring should use is marked `@Autowired`.)

> **Why bother.** A class that receives its collaborators can be given fakes. The tests build an `EventDiscovery` with a mock client, a mock Kafka template and a fake clock, and check exactly which events it asks for, with no network and no broker.

## 3.2 The two ways a class becomes a bean

| How | Meaning | Used for |
|---|---|---|
| **Stereotype annotation on the class** `@Component`, `@Service`, `@Repository`, `@RestController` | Component scanning finds the class and constructs it. The four differ mostly in what they tell the reader. `@Repository` also translates database exceptions into Spring's own; `@RestController` also makes the methods answer HTTP with JSON. | Almost every class: `EventDiscovery`, `MetaService`, `TeamStore`, `MetaController` and so on. |
| **`@Bean` method in a `@Configuration` class** | Spring calls the method once at start-up and keeps what it returns as a bean named after the method. The method's parameters are injected like a constructor's. | Objects that need arguments or set-up, or whose class isn't ours to annotate: the `Clock`, the Kafka `NewTopic`s, the error handler, the two filter registrations, the `TokenService`, the `OpenAPI` description. |

### Every `@Bean` in the project

| Where | Method | What the bean is, and who uses it |
|---|---|---|
| `BackendApplication` | `clock()` | The system clock in UTC. Injected into discovery, the fetcher, the ladder discovery, `MetaService` and the token service, so tests can replace it. |
| `Topics` | `tournamentsDiscovered()`, `standingsFetched()`, `pairingsFetched()`, `pokemonUsage()`, `ladderStats()` and four `...Dlt()` | Nine `NewTopic` descriptions. Nothing injects them: `KafkaAdmin` collects every `NewTopic` bean and creates the topics on the broker at start-up. |
| `KafkaErrorHandling` | `kafkaErrorHandler(template)` | The retry-then-dead-letter policy. Spring Boot applies the one `CommonErrorHandler` bean to every listener. |
| `MetaWebConfig` | `v1Etags()` | Registers the ETag filter on `/v1/*`. |
| `SecurityConfig` | `tokenService(properties, clock)` | The one `TokenService`, holding the signing secret. Used by the filter and `AuthController`. |
| `SecurityConfig` | `apiAuth(properties, tokens)` | Registers `ApiAuthFilter` on every path, first in order. |
| `SecurityConfig` | `openApi(properties)` | Tells Swagger UI about the two credentials when signing in is on. |

## 3.3 The annotations on `BackendApplication`

- `@SpringBootApplication` combines three: `@Configuration` (may hold `@Bean` methods), `@ComponentScan` (find annotated classes in

this package and below) and `@EnableAutoConfiguration` (configure whatever libraries are on the classpath from `application.yml`: the data source, Flyway, the Kafka producer and listener factory, Jackson, Tomcat).

- `@ConfigurationPropertiesScan` finds the settings records (next section).
- `@EnableScheduling` turns on `@Scheduled`.

## 3.4 Settings: `@ConfigurationProperties`

Three records hold the project's own settings: `LimitlessProperties` (`pkref.limitless`), `SmogonProperties` (`pkref.smogon`) and `ApiAuthProperties` (`pkref.api`). Spring builds each from the matching block of `application.yml`, converting text to the field's type: `30m` to a `Duration`, `M-C,M-B` to a `List<String>`. Kebab-case in YAML (`poll-interval`) matches camelCase in Java (`pollInterval`).

In the YAML, `${LIMITLESS_LOOKBACK:7d}` means "the environment variable `LIMITLESS_LOOKBACK` if set, otherwise 7d". Each record's compact constructor fills in defaults and, for `ApiAuthProperties`, refuses bad values, so a misconfigured server fails at start-up with a clear message.

## 3.5 Timers: `@Scheduled`

`EventDiscovery.poll()` and `LadderDiscovery.poll()` are called by Spring's scheduler. `fixedDelay` measures from the end of one run to the start of the next, so a slow run can't overlap itself. The scheduler's thread pool is set to 2 in `application.yml` so the two polls don't wait for each other.

`@ConditionalOnProperty(name = "pkref.limitless.ingest-enabled", ...)` on the class means the bean is only created when that setting is true. Tests switch both polls off that way.

## 3.6 Kafka: `KafkaTemplate` and `@KafkaListener`

**Producing.** `KafkaTemplate<String, Object>` is auto-configured from `spring.kafka.producer`. `kafka.send(topic, key, value)` serializes the value to JSON and returns a future; the project always passes that future to `KafkaSends.await`, so a failed send throws where it happened instead of being logged and forgotten.

**Consuming.** A method annotated `@KafkaListener(topics = ..., groupId = ...)` gets a consumer thread of its own. Spring polls the topic, rebuilds each record's value from JSON into the parameter's type, and calls the method. Returning normally means "done"; throwing means "failed", which hands the record to the error handler. There are five listener methods:

| Method | Topic | Group |
|---|---|---|
| `EventFetcher.onDiscovered` | tournaments.discovered | standings-fetcher |
| `TeamStoreConsumer.onStandings` | standings.fetched | team-store-standings |
| `TeamStoreConsumer.onPairings` | pairings.fetched | team-store-pairings |
| `UsageConsumer.onStandings` | standings.fetched | usage-aggregator |
| `LadderStoreConsumer.onStats` | ladder.stats | ladder-store |

## 3.7 The web layer

- `@RestController`: a bean whose methods handle HTTP requests; return values become JSON.
- `@RequestMapping("/v1")` on the class: the path prefix for all its methods.
- `@GetMapping("/formats/{format}/pokemon")`, `@PostMapping("/token")`: the verb and path of one method.
- `@PathVariable`: an argument taken from a `{...}` part of the path. `@RequestParam`: one taken from the query string, required unless it has `required = false` or a `defaultValue`.
- `ResponseEntity<T>`: a response whose status and headers the method sets, used here to add `Cache-Control`.
- `@ExceptionHandler(X.class)`: a method of the same controller that Spring calls when a handler throws `X`. The project maps `IllegalArgumentException` to 400 and `NoSuchElementException` to 404, so services can signal "bad input" and "not there" with ordinary exceptions.
- **Filters** (`FilterRegistrationBean`): code that runs around every matching request, before any controller. Two here: `ApiAuthFilter` on `/*`, first, and `ShallowEtagHeaderFilter` on `/v1/*`.
- `@Tag`, `@Operation`, `@Parameter`, `@Schema`: springdoc's. They describe the API for Swagger UI and have no effect on behavior.

## 3.8 The database layer

- `JdbcTemplate` (and `NamedParameterJdbcTemplate`): run SQL with parameters and map rows with a lambda. Used by `TeamStore`, `MetaRepository`, `CorpusRepository`, `EventFetchRepository`, `LadderStore`.
- **JPA**: `@Entity` classes mapped to tables by Hibernate, with repositories Spring Data generates from interfaces (`UsageCounterRepository`). Used only by the original usage counters.

- `@Transactional` : the method runs in one database transaction, committed when it returns and rolled back if it throws. On `TeamStore.putStandings` , `putPairings` , `LadderStore.put` and `UsageAggregationService.apply` . It works through a proxy Spring puts around the bean, so it applies to calls coming from another bean, which all of these are.

## 3.9 Java features worth being able to explain

- **Records** ( `public record Share(String value, double share) {}` ): immutable data classes. All wire types, responses, settings and query rows are records. A **compact constructor** ( `public LimitlessProperties { ... }` ) validates or defaults the fields.
- **Text blocks** ( `"""..."""` ): multi-line strings, used for SQL.
- **`var`** : a local variable whose type the compiler infers.
- **Switch expressions**: `return switch (result) { case "P1" -> ...; default -> 1; };`
- **Streams and lambdas**: most of `MetaService` 's counting and sorting.
- **`Optional`** : a return type that says "there may be none" ( `LadderStore.find` ).

# Part 4. Kafka, as this project uses it

## 4.1 The ideas

**Topic**
A named, append-only log of records. Producers add to the end; consumers read forward from wherever they are. Reading does not remove anything.

**Record**
A key, a value and some headers. Here the key is always a string (an event id, or `format|month|rating`) and the value is JSON.

**Partition**
A topic is split into partitions, each an ordered log. A record's key decides its partition (by hash), so records with the same key stay in order. Each topic here has 3.

**Offset**
A record's position in its partition. A consumer's progress is "the next offset to read".

**Consumer group**
A name under which consumers share progress. Kafka stores each group's offsets. Two groups reading one topic each see every record, independently; two consumers in one group split the partitions between them.

**Compaction**
A cleanup policy that, instead of deleting old records by age, keeps at least the newest record of each key. A compacted topic behaves like a table of "latest value by key" that can still be read from the beginning.

**Dead letter topic (DLT)**
A topic where records are parked after repeated failure, so they can be inspected and the rest of the topic isn't blocked.

**KRaft**
Kafka running its own metadata quorum, with no ZooKeeper. This project runs one node that is both broker and controller.

## 4.2 The topics

| Topic | Key | Value | Written by | Read by (group) | Kept |
|---|---|---|---|---|---|
| `tournaments.discovered` | event id | `TournamentDiscovered` | `EventDiscovery` | `EventFetcher` (standings-fetcher) | Broker default (7 days): a work queue |
| `standings.fetched` | event id | `StandingsFetched` | `EventFetcher` | `TeamStoreConsumer` (team-store-standings), `UsageConsumer` (usage-aggregator) | Compacted, for good |
| `pairings.fetched` | event id | `PairingsFetched` | `EventFetcher` | `TeamStoreConsumer` (team-store-pairings) | Compacted, for good |
| `pokemon.usage` | `format|species` | `UsageUpdated` | `UsageConsumer` | Nothing yet | Compacted |
| `ladder.stats` | `format|month|rating` | `Ladder.StatsFetched` | `LadderDiscovery` | `LadderStoreConsumer` (ladder-store) | Compacted, for good |
| `*.DLT` (four) | as the original | as the original, plus failure headers | The error handler | Nothing: for people to look at | Broker default |

The fetched-data topics accept records up to 5 MB (`max.message.bytes`), because a large event's standings are over the 1 MB default, and the producer compresses with zstd.

## 4.3 Why Kafka at all

Not for volume: a few hundred events a month would fit in a queue table. Kafka is here as a **durable, replayable log**:

- **Replay.** The compacted topics hold every event ever fetched. If a table is lost, or a rule changes (how names are standardized, what counts as a species), the tables are rebuilt by resetting a consumer group to the beginning. Limitless doesn't have to be crawled again.
- **Independent readers.** The team store and the usage counters both read `standings.fetched` without knowing about each other. Adding a third reader is a new group id.
- **Decoupling slow from fast.** Discovery takes seconds; fetching is throttled and can take hours during a backfill. The queue between them survives restarts.
- **Failure isolation.** A record that can't be processed is retried and then set aside, without stopping the others.

The honest cost: a broker is more to run than this volume needs. The plan calls that out, and keeps it to a single KRaft node.

## 4.4 Delivery guarantees

**Producer.** `acks=all` and `enable.idempotence=true`: a send succeeds only when the broker has the record, and a retried send isn't stored twice. Every send is awaited, so "published" means confirmed.

**Consumer.** Spring Kafka commits offsets only after the listener has returned for the records it was given. If the process dies before the commit, those records are delivered again. That is **at-least-once** delivery: never lost, sometimes repeated.

**So every consumer is idempotent**: handling a record twice ends in the same state as handling it once.

- `TeamStore` deletes an event's rows and inserts them again, in one transaction, and ignores a fetch older than the one stored (by `fetchedAt`).
- `LadderStore` replaces a row unless the stored one is newer.
- `UsageAggregationService` can't "replace" a counter, so it records each processed event's id in the same transaction as the counts and skips an id it has seen.
- `EventFetcher` repeating a fetch costs three requests and republishes; compaction and the stores absorb the duplicate.

At-least-once delivery plus idempotent handling gives the effect people mean by "exactly once", without Kafka transactions.

## 4.5 The settings that matter (`application.yml`)

| Setting | Value | Why |
|---|---|---|
| `producer.acks` | `all` | Don't count a send until it's durable. |
| `enable.idempotence` | `true` | Retries can't duplicate. |
| `compression-type` | `zstd` | Standings are repetitive JSON. |
| `max.request.size` | 5 MB | Large events' standings. |
| `value-serializer` / `-deserializer` | JSON | Records travel as JSON with a type header naming the Java record. |
| `spring.json.trusted.packages` | `...backend.model` | The deserializer will only build classes from that package, whatever a record's header says. |
| `auto-offset-reset` | `earliest` | A new or reset group reads a topic from its start: that is what makes rebuilding work. |
| `max-poll-records` | 10 | The fetcher can wait minutes per request under the throttle. A consumer silent for longer than `max.poll.interval.ms` is presumed dead and its records reassigned, so: few records a poll, 15 minutes allowed between polls. |
| `max.poll.interval.ms` | 900000 | |
| `admin.modify-topic-configs` | `true` | Changes in `Topics.java` are applied to existing topics. |

## 4.6 Errors: retry, then dead-letter

`KafkaErrorHandling` defines one `DefaultErrorHandler`: when a listener throws, retry the record up to 3 more times, 2 seconds apart; if it still fails, publish it to `<topic>.DLT` on the same partition and continue with the next record.

# Part 5. The database

## 5.1 Tables

| Table | One row is | Written by | Read by | Migration |
|---|---|---|---|---|
| `event_fetch` | what was asked for and fetched of one event | `EventFetchRepository` | `EventDiscovery` (through the repository) | V2 |
| `event` | one stored tournament | `TeamStore` | `MetaRepository`, `CorpusRepository` | V2 |
| `event_phase` | one phase of an event (Swiss, top cut) | | | V2 |
| `team` | one player's entry: placing and record | | | V2 |
| `team_member` | one Pokémon on a team, with its species key and Mega Stone | | | V2, V3 |
| `team_member_move` | one move of one member | | | V2 |
| `pairing` | one match, with its result | | | V2 |
| `pairing_fetch` | when an event's pairings were fetched | `TeamStore` | `TeamStore` | V2 |
| `store_version` | the single counter every write bumps | `TeamStore` | `MetaRepository.version` | V4 |
| `usage_counter` | one all-time count | `UsageAggregationService` (JPA) | `UsageController` | V1 |
| `processed_tournament` | an event already counted | | `UsageAggregationService` | V1 |
| `ladder_stats` | one reduced month of a format at a rating, as JSON text | `LadderStore` | `LadderStore`, for `LadderService` | V5 |

Keys worth knowing: a team is `(event_id, player)`; a member adds `slot`; a move adds `move_slot`. Deleting an `event` cascades to its phases, teams, members and moves (`on delete cascade`). `pairing` has no foreign key to `event`, because an event's pairings can arrive before its standings.

## 5.2 Flyway

The schema lives in `src/main/resources/db/migration` as `V1__...sql` to `V5__...sql`. At start-up Flyway runs any file that hasn't run on this database, in number order, and records each in its own table, `flyway_schema_history`, with a checksum.

- To change the schema, **add a new file** with the next number. Never edit one that has been applied: Flyway compares checksums at start-up and refuses to start on a mismatch. (This is why the comment pass that produced this guide left the SQL files alone.)
- Hibernate is set to `ddl-auto: validate`: it creates nothing, and only checks that the two entity classes match their tables.
- Tests run the same migrations on H2 (in memory) and, where Docker is available, on real Postgres 17 through Testcontainers, because H2 accepts some SQL that Postgres rejects.

## 5.3 Two styles of data access, and why

The first version of the backend used **JPA**: annotate a class as an `@Entity`, declare a repository interface, and let Spring Data generate the queries. It's concise for "load a few objects, change them, save them", which is what the counters do.

Everything since uses **`JdbcTemplate` and plain SQL**. The team store's work is "delete an event's rows and bulk-insert a few thousand", and the insights' work is reporting queries over joins. Both read more plainly as SQL, batch well, and behave the same on H2 and Postgres. The counting itself is done in Java, not SQL, so definitions live in one tested place (`MetaService`).

## 5.4 Transactions and the version counter

`TeamStore.putStandings` is `@Transactional`: delete, bump `store_version`, insert, all or nothing. A reader on another connection sees either the old event or the new one, never half.

`MetaService` caches each computed answer under a key that includes `store_version`. Because the bump commits in the same transaction as the rows, a reader sees a new version exactly when it can see the new rows, so a cached answer is never served for data it wasn't computed from.

# Part 6. Signing in

The server is meant for one owner's devices, reached over a private network (Tailscale). By default it listens on `127.0.0.1` only and lets every request in. When API keys are configured, every request must be signed in. The whole mechanism is five classes in `security/`; Spring Security isn't used.

## 6.1 Two credentials

| | API key | Access token |
|---|---|---|
| What it is | `pkr_` plus 32 random bytes, base64url. Made by `scripts/new-api-key.sh`. | A JWT signed with HMAC-SHA256. |
| Lives | Until its hash is removed from the server's settings | 15 minutes |
| Accepted at | Only `POST /v1/auth/token` | Everything else |
| What the server stores | Only its SHA-256 hash (`PKREF_API_KEY_HASHES`) | Nothing: it verifies the signature |

> **Why two.** The long-lived secret travels once every 15 minutes instead of with every request, and a token that leaks stops working by itself. Storing only the key's hash means the server's configuration, backups and logs never contain a usable key.

## 6.2 The flow

1. The app sends `POST /v1/auth/token` with `Authorization: Bearer <key>`.
2. `ApiAuthFilter` hashes the key and compares it with every configured hash, each in constant time (`MessageDigest.isEqual`). On a match it stores the key's short id (the first 8 hex digits of its hash) on the request and lets it through.
3. `AuthController.token` asks `TokenService.issue(keyId)` for a token and returns it with `Cache-Control: no-store`.
4. For every other request the app sends `Authorization: Bearer <token>`, and the filter calls `TokenService.verify`.
5. When a token is refused with `token_expired` or `invalid_token`, the app trades its key again and repeats the request once.

## 6.3 What a token contains

Three base64url parts joined by dots: header, claims, signature. The first two are readable by anyone; the signature is what makes them trustworthy.

| Claim | Value | Checked on verify |
|---|---|---|
| `alg` (header) | `HS256` | Must be exactly HS256, checked before the signature |
| `iss` | `pkreference-backend` | Must match |
| `aud` | `pkreference-api` | Must contain it |
| `sub` | the key's id | Must be a key that is still configured |
| `iat`, `exp` | issued at, expires at | `exp` must be in the future |
| `jti` | a random id | Not checked; makes each token unique |

## 6.4 The order of checks in `TokenService.verify`, and why

1. **Parse.** Not three dot-separated parts: `MALFORMED`.
2. **Algorithm.** The header is written by whoever sent the token, so the server decides the algorithm. This closes the classic JWT attacks: `alg: none` (no signature) and algorithm switching.
3. **Signature.** HMAC-SHA256 with the server's secret. Only after this are the claims the server's own words.
4. **Expiry.**
5. **Issuer, audience, subject.** A token signed by this server for something else is not accepted.
6. **Key still configured.** Removing a key's hash from the settings ends its tokens too.

## 6.5 Details an interviewer may probe

- **The signing secret** is 32 random bytes generated at start-up unless `PKREF_TOKEN_SECRET` is set. A restart therefore ends all tokens; clients just trade their key again.
- **Credentials are never read from the URL** (where proxies and logs keep them) and never logged. The refusal log line names the path and the reason only.

- **Swagger UI's paths are open**, but a path that the server would rewrite (containing `..`, `%`, `;` or `//`) never counts as documentation.
- **A token can't be traded for a token.** The token path only accepts a key, and `AuthController` checks again that the filter matched one.
- **Caching.** Signed-in answers have `Cache-Control: public` rewritten to `private`, so a shared cache can't serve them to someone else.
- **What this is not.** No users, roles or sessions; no refresh tokens (the key plays that part); no token revocation list (a 15-minute life and key removal stand in for it). A public deployment would need more, as the plan says.

# Part 7. The API

All read-only except the token request. JSON, cached for 15 minutes, with weak ETags on `/v1`. With the server running, Swagger UI at `/swagger-ui.html` documents every field.

| Request | Controller method | Returns |
|---|---|---|
| `GET /v1/formats` | `MetaController.formats` | Each stored format's events, teams and dates |
| `GET /v1/formats/{f}/pokemon?window=` | `MetaController.pokemon` | Every Pokémon's usage, top-cut usage, trend and win record |
| `GET /v1/formats/{f}/pokemon/{key}` | `MetaController.pokemon` | One Pokémon's items, abilities, natures, moves, Mega Stones, teammates, sets, weekly usage |
| `GET /v1/formats/{f}/cores` | `MetaController.cores` | Pairs and trios chosen together, with lift |
| `GET /v1/formats/{f}/archetypes` | `MetaController.archetypes` | Archetypes and their records against each other |
| `GET /v1/formats/{f}/archetypes/{id}` | `MetaController.archetype` | One archetype's best-placed teams |
| `GET /v1/formats/{f}/events?limit=` | `MetaController.events` | The newest events and their winners |
| `GET /v1/formats/{f}/tournaments?page=&limit=` | `CorpusController.tournaments` | Stored events in Limitless's shape |
| `GET /v1/tournaments/{id}/standings` | `CorpusController.standings` | One event's standings and teams in Limitless's shape |
| `GET /v1/ladder` | `LadderController.index` | Which ladder months are stored |
| `GET /v1/ladder/{format}?rating=&month=` | `LadderController.stats` | One month of ladder stats, with trends and usage by rating |
| `POST /v1/auth/token` | `AuthController.token` | An access token for an API key |
| `GET /api/usage?format=` | `UsageController.top` | (Old) all-time usage |
| `GET /api/usage/{species}?format=` | `UsageController.detail` | (Old) one Pokémon's all-time counts |

**Status codes.** 200 with a body; 304 when `If-None-Match` matches; 400 for a bad parameter (`IllegalArgumentException`); 401 with a `code` field when signing in fails; 404 for something not stored (`NoSuchElementException`).

**Windows.** `window=14d`, `30d` (the default) or `regulation` (every stored event of the format).

# Part 8. File by file

Every source file, in the order the data flows through them. For each: what the class is, its annotations, and every method with what it does. This part is generated from the comments in the code itself, so what you read here is what you will see when the file is open in front of you.

## The application

One class: where the program starts and what Spring builds from.

### BackendApplication.java

`src/main/java/com/pkreference/backend/BackendApplication.java`

```
@SpringBootApplication @ConfigurationPropertiesScan @EnableScheduling public class BackendApplication
```

The program's entry point, and the root of everything Spring builds.

How Spring starts this server: `main` hands this class to `SpringApplication.run`, which creates the "application context", the container that makes one object (a "bean") of each class below and passes each the other beans its constructor asks for. Nothing in this project calls `new` on a controller, a repository or a Kafka listener: Spring does, in dependency order.

The three annotations:

- `@SpringBootApplication` is three in one. `@Configuration`: this class may declare beans with `@Bean` methods. `@ComponentScan`: look through this package and every package under it for classes marked `@Component`, `@Service`, `@Repository`, `@RestController` or `@Configuration`, and make a bean of each. `@EnableAutoConfiguration`: look at what's on the classpath (Tomcat, Kafka, a JDBC driver, Flyway) and configure it from application.yml, which is why there is no code here that opens a database connection or creates a Kafka producer.
- `@ConfigurationPropertiesScan` finds the `@ConfigurationProperties` records (LimitlessProperties, SmogonProperties, ApiAuthProperties) and fills them from application.yml and environment variables.
- `@EnableScheduling` switches on `@Scheduled` methods: without it, EventDiscovery's and LadderDiscovery's polls would never run.

Start-up, roughly in the order the logs show: Flyway runs the SQL migrations, the topics in Topics are created on the broker, the Kafka listeners subscribe, Tomcat starts listening, and after their initial delays the two scheduled polls begin.

#### MEMBERS

```
public static void main(String[] args)
```

Starts Spring Boot: builds the application context and starts the embedded Tomcat web server on port 8080. It returns once the server is up; the server's own threads keep the program running. `args` are the command line's, which can override any setting (`--server.port=9090`).

```
@Bean Clock clock()
```

The time, as discovery, the fetcher and the insights read it. A `@Bean` method's return value becomes a bean: any constructor with a `Clock` parameter gets this one object. Asking Spring for the clock, rather than calling `Instant.now()`, is what lets a test put a fixed or movable clock in its place and check behavior at an exact moment.

## config: set-up shared by everything

The Kafka topics, what happens when a listener fails, the helper that makes sends throw, and the two settings records for the sources.

### KafkaErrorHandling.java

`src/main/java/com/pkreference/backend/config/KafkaErrorHandling.java`

```
@Configuration public class KafkaErrorHandling
```

What happens when a `@KafkaListener` method throws.

By default Spring Kafka retries a failed record a few times and then skips it, logging the error: the data is lost quietly. This class replaces that with "retry, then park it where it can be found". Spring Boot gives the one `CommonErrorHandler` bean in the context to every listener container it builds, so declaring the bean is all the wiring there is.

#### MEMBERS

```
@Bean CommonErrorHandler kafkaErrorHandler(KafkaOperations<Object, Object> template)
```

Retry 3 times, 2 seconds apart (four attempts in all), then publish the record to `<topic>.DLT`, same partition, and move on. The dead-letter record carries the original value plus headers saying which exception failed it.

`template` is Spring Boot's auto-configured KafkaTemplate, injected as this method's parameter: a `@Bean` method's parameters are filled in from the context like a constructor's.

## KafkaSends.java

src/main/java/com/pkreference/backend/config/KafkaSends.java

```
public final class KafkaSends
```

Waits for a Kafka send, so a failure (a record over the topic's size limit, the broker down) throws where it happened. KafkaTemplate.send only returns a future: unread, a failed send was logged and the event lost, while the listener committed its offset.

### MEMBERS

```
public static <T> T await(CompletableFuture<T> send)
```

Blocks until the broker has acknowledged the send, and returns its result; throws `IllegalStateException` if it failed or took over 30 seconds. Called as `KafkaSends.await(kafka.send(topic, key, value))`. Throwing matters inside a `@KafkaListener`: the exception makes Spring retry the record being handled instead of committing past it.

## LimitlessProperties.java

src/main/java/com/pkreference/backend/config/LimitlessProperties.java

```
@ConfigurationProperties(prefix = "pkref.limitless") public record LimitlessProperties(String baseUrl, String game,
List<String> formats, int pageSize, boolean ingestEnabled, Duration pollInterval, int requestsPerWindow, Duration window,
int minPlayers, Duration lookback, int maxPages, Duration settleAfter, Duration refetchAfter, Duration pendingTimeout)
```

Limitless and how events are found. minPlayers, settleAfter and refetchAfter match the app's TeamCorpusConfiguration, so the server keeps the events Team Search would.

`@ConfigurationProperties(prefix = "pkref.limitless")` tells Spring to build this record from the settings under `pkref.limitless` in application.yml: `poll-interval: 30m` becomes `pollInterval`, a `Duration`; `formats: M-C,M-B` becomes a list. Each setting there can name an environment variable (`${LIMITLESS_LOOKBACK:7d}`), which is how a deployment changes it without a rebuild. The record is then a bean like any other: classes that need the settings take a `LimitlessProperties` in their constructor.

A record is immutable, so the settings can't change while the server runs.

### FIELDS AND CONSTANTS

```
public LimitlessProperties
```

The compact constructor: it runs on every construction, before the fields are set, and fills in a default for anything left out (null, or zero for a number). So the defaults live here, in one place, and a test can pass nulls and zeros for "the default".

### MEMBERS

```
public static LimitlessProperties defaults(String game, List<String> formats)
```

Every default, for one game and its formats.

## SmogonProperties.java

src/main/java/com/pkreference/backend/config/SmogonProperties.java

```
@ConfigurationProperties(prefix = "pkref.smogon") public record SmogonProperties(String baseUrl, boolean ingestEnabled,
Duration pollInterval, int lookbackMonths, List<Integer> ratings, Duration requestGap, String userAgent)
```

Smogon's monthly ladder stats and how they're fetched: the settings under `pkref.smogon` in application.yml, bound into this record by Spring as LimitlessProperties is.

### FIELDS AND CONSTANTS

```
public SmogonProperties
```

Fills in a default for anything left out, as LimitlessProperties' compact constructor does.

### MEMBERS

```
public static SmogonProperties defaults()
```

Every default, with fetching on: what tests build a client or a discovery with.

## Topics.java

src/main/java/com/pkreference/backend/config/Topics.java

```
@Configuration public class Topics
```

The Kafka topics, declared in code. Each `@Bean` method below returns a `NewTopic`; at start-up Spring's `KafkaAdmin` asks the broker for each one and creates those that don't exist. With `spring.kafka.admin.modify-topic-configs` on (application.yml), a setting changed here also reaches a topic that already exists.

A topic is an append-only log of records, each a key and a value. The pipeline, in topic order:

- `tournaments.discovered`: "fetch this event". Written by EventDiscovery, read by EventFetcher. A work queue: its records are only instructions, so it isn't kept for ever.
- `standings.fetched` and `pairings.fetched`: what Limitless answered. Written by EventFetcher, read by TeamStoreConsumer and UsageConsumer. These are the system's record of the truth: kept for good, so any table can be rebuilt by reading them again.
- `pokemon.usage`: the old counters' latest row per Pokémon, written by UsageConsumer.
- `ladder.stats`: Smogon's monthly stats, reduced. Written by LadderDiscovery, read by LadderStoreConsumer.
- The `.DLT` topics ("dead letter topics"): where a record goes after its listener has failed on it four times (KafkaErrorHandling), so one bad record doesn't block the rest.

Three words used below. **Partitions**: a topic is split into this many logs; records with the same key always land in the same one, so they keep their order. **Replicas**: copies of each partition on other brokers; 1 here, because there is one broker. **Compaction**: instead of deleting old records by age, the broker keeps at least the newest record of every key, which turns a topic into a table of "latest value by key" that can still be read from the start.

#### FIELDS AND CONSTANTS

```
public static final String TOURNAMENTS_DISCOVERED = "tournaments.discovered"
```

The topics' names, as constants: producers and `@KafkaListener`s refer to these, so a name can't be misspelled in one place.

```
private static final int PARTITIONS = 3
```

Three partitions a topic. More partitions than one lets several consumers of one group share the work; with one consumer a listener, as here, it reads all three.

```
static final String MAX_MESSAGE_BYTES = "5242880"
```

The largest record a fetched-data topic takes, as the producer's max.request.size. The broker's default is about 1 MB, which a large event's standings (about 1.2 KB a player, before compression) can pass.

#### MEMBERS

```
@Bean NewTopic tournamentsDiscovered()
```

The work queue between discovery and the fetcher. Default settings: records are deleted after the broker's retention time (7 days), which is fine for instructions that are acted on in minutes.

```
@Bean NewTopic standingsFetched()
```

The record of every event fetched, kept for good: compacted to the latest standings per tournament id and never deleted, so any count can be rebuilt by reading it again.

```
@Bean NewTopic tournamentsDiscoveredDlt()
```

Where a "fetch this event" record goes when the fetcher keeps failing on it (Limitless down, say). Discovery asks for the event again later, so nothing has to read this topic; it's there to be looked at.

```
@Bean NewTopic pairingsFetched()
```

Every event's matches, kept for good as standings.fetched is.

```
@Bean NewTopic pairingsFetchedDlt()
```

Takes the same records as pairings.fetched, with the failure in their headers.

```
@Bean NewTopic standingsFetchedDlt()
```

Takes the same records as standings.fetched, with the failure in their headers.

```
@Bean NewTopic ladderStats()
```

Smogon's monthly ladder stats, reduced (Ladder.Stats, about 1 MB at most), kept for good: the latest per format, month and rating. Smogon's own files, up to 7 MB, aren't kept; Smogon keeps every month.

```
@Bean NewTopic ladderStatsDlt()
```

Takes the same records as ladder.stats, with the failure in their headers.

```
@Bean NewTopic pokemonUsage()
```

Latest usage row per key (format|species) survives compaction, and a compacted topic doesn't delete by age. The counters themselves are in Postgres; this topic is a feed of their changes, for anything that wants to follow them.

## model: the shapes of the data

Records only. They define the JSON read from Limitless, carried on the Kafka topics and, for the ladder, stored and served.

### Events.java

src/main/java/com/pkreference/backend/model/Events.java

```
public final class Events
```

Wire types: the shapes of the JSON that travels between Limitless, the Kafka topics and the app. Mirrors PKReference/LimitlessAPI.swift; fields the API omits are nullable.

Each is a Java `record`: an immutable class whose constructor, accessors (`tournament.id()`), `equals` and `toString` the compiler writes from the one line of fields. Jackson, the JSON library Spring uses, reads and writes records by their field names, so these one-liners are the whole definition of the JSON.

They're used three ways: to read Limitless's answers (LimitlessClient), as the values of the Kafka records (the last four records below, serialized to JSON by the producer and back by each listener), and to answer the app in Limitless's own shapes (CorpusController).

`@JsonIgnoreProperties(ignoreUnknown = true)`, on the records read from outside: a field in the JSON that the record doesn't have is skipped instead of failing the read, so Limitless adding a field, or a newer server having written a record with one, doesn't break this one.

The class itself is only a namespace: `final` with a private constructor, never created.

#### MEMBERS

```
@JsonIgnoreProperties(ignoreUnknown = true) public record Tournament(String id, String name, String game, String format, String date, int players)
```

One event as Limitless lists it (GET /tournaments). `date` is an ISO-8601 instant kept as the text Limitless sent ("2026-10-05T18:00:00.000Z"); FetchPolicy.startOf parses it.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record TournamentDetails(Organizer organizer, String platform, Boolean decklists, Boolean isOnline, List<Phase> phases)
```

GET /tournaments/{id}/details.

> ```
> @JsonIgnoreProperties(ignoreUnknown = true) public record Organizer(Integer id, String name)
> ```
>
> Who ran the event.
>
> ```
> @JsonIgnoreProperties(ignoreUnknown = true) public record Phase(int phase, String type, Integer rounds, String mode)
> ```
>
> type is SWISS, SINGLE_BRACKET and so on; mode is BO1, BO3 or BO5.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record Standing(String player, String name, String country, Integer placing, Record record, Deck deck, List<TeamMember> decklist, Integer drop)
```

GET /tournaments/{id}/standings: one player's entry. `player` is the username, unique in an event; `decklist` is the team, null when the event didn't publish teams; `drop` is the round the player dropped in, null when they didn't.

> ```
> @JsonIgnoreProperties(ignoreUnknown = true) public record Record(int wins, int losses, int ties)
> ```
>
> A player's wins, losses and ties.
>
> ```
> @JsonIgnoreProperties(ignoreUnknown = true) public record Deck(String id, String name, List<String> icons)
> ```
>
> Limitless's label for a team; read, but not kept.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record TeamMember(String name, String limitlessId, String item, String ability, List<String> attacks, String nature, String tera)
```

One Pokémon on a team, as the player typed it: names here are free text, which is why NameStandardizer and SpeciesVocabularies exist. `@JsonProperty("id")` maps Limitless's JSON field "id" (its slug for the species, "arcanine-hisui") to a field with a clearer name.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record Pairing(int round, int phase, Integer table, String match, String player1, String player2, String winner)
```

GET /tournaments/{id}/pairings: one match. winner is the winner's username, "0" for a tie or "-1" for a double loss (Limitless sends the numbers as numbers). With no player2 it's a bye (winner is player1) or a loss for not showing up (winner is -1). table is null in live brackets, which label their matches instead ("T16-8").

```
public record TournamentDiscovered(Tournament tournament)
```

Topic tournaments.discovered, key = tournament id: fetch this event.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record StandingsFetched(Tournament tournament, List<Standing>
standings, TournamentDetails details, Instant fetchedAt, Boolean standingsFinal)
```

Topic standings.fetched, key = tournament id. details, fetchedAt and standingsFinal are null in records from before they were added; those count as final.

> ```
> public StandingsFetched(Tournament tournament, List<Standing> standings)
> ```
>
> A second constructor, for tests and old callers: standings alone, with nothing known about the details, the time or whether they're final.
>
> ```
> public boolean isFinal()
> ```
>
> Whether these standings won't change: fetched long enough after the event started.

```
public record PairingsFetched(Tournament tournament, List<Pairing> pairings, Instant fetchedAt)
```

Topic pairings.fetched, key = tournament id.

```
public record UsageUpdated(String format, String species, long teams, long totalTeams)
```

Topic pokemon.usage, key = "format|species".

### Ladder.java

`src/main/java/com/pkreference/backend/model/Ladder.java`

```
public final class Ladder
```

Ladder stats, reduced from one of Smogon's monthly files to what the app shows (BackendIntegration-PHASE5.md §3). The app's SmogonLadder.reduce makes the same record from the same files; golden/ladder-fixture.json keeps the two equal.

As Events' records are, these are wire types: `Stats` is both the value of a `ladder.stats` Kafka record (inside `StatsFetched`) and what ladder_stats stores as JSON text. The `@Schema` annotations change nothing at run time: springdoc reads them to describe each field in Swagger UI.

#### MEMBERS

```
@JsonIgnoreProperties(ignoreUnknown = true) public record Share(String value, double share)
```

A value, and the share of the Pokémon's teams with it, 0 to 1.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record Pokemon(String name, String key, String megaStone, double
usage, Double leadShare, List<Share> abilities, List<Share> items, List<Share> moves, List<Share> natures, List<Share>
spreads, List<Share> teammates)
```

One Pokémon's month. `Double leadShare` is the boxed type because it can be absent (null); `double usage` can't.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record Stats(int reduction, String format, String regulation, String
mode, int bestOf, String month, int rating, long battles, List<Pokemon> pokemon)
```

One of Smogon's files, reduced: a format's month at one rating.

```
@JsonIgnoreProperties(ignoreUnknown = true) public record StatsFetched(Stats stats, Instant fetchedAt)
```

Topic ladder.stats, key = "format|month|rating".

> ```
> public String key()
> ```
>
> The Kafka record's key: one key per file, so the compacted topic keeps the newest record of each.
>
> ```
> public static String key(String format, String month, int rating)
> ```
>
> "format|month|rating".

## ingest: from Limitless to Kafka

The first half of the write path for tournaments: decide what to fetch, fetch it without breaking the rate limit, publish it.

### EventDiscovery.java

`src/main/java/com/pkreference/backend/ingest/EventDiscovery.java`

```
@Component @ConditionalOnProperty(name = "pkref.limitless.ingest-enabled", havingValue = "true", matchIfMissing = true)
public class EventDiscovery
```

Walks each format's tournament list back through the lookback and asks for the events to fetch (FetchPolicy) on tournaments.discovered. A long lookback, set once, is the backfill.

This is where the pipeline starts, and the only class that runs on a timer rather than in answer to a request or a record. It doesn't fetch events itself: it publishes one small "fetch this event" record for each, and EventFetcher does the slow part. Splitting the two means a poll finishes in seconds, a failed fetch retries on its own, and the queue survives a restart.

`@ConditionalOnProperty` : the bean exists only when `pkref.limitless.ingest-enabled` is true (or unset). Tests set it to false, so no test polls Limitless.

**MEMBERS**

```
public EventDiscovery(LimitlessClient client, EventFetchRepository fetches, KafkaTemplate<String, Object> kafka,
LimitlessProperties props, Clock clock)
```

Constructor injection: Spring sees one constructor and supplies each parameter from the context. `KafkaTemplate<String, Object>` is Spring Boot's ready-made producer, with String keys and values serialized to JSON (application.yml).

```
@Scheduled(initialDelayString = "${pkref.limitless.initial-delay:5s}", fixedDelayString = "${pkref.limitless.poll-
interval:30m}") public void poll()
```

Run by Spring's scheduler: first 5 seconds after start-up, then again 30 minutes after each run ends ( `fixedDelay` counts from the end of a run, so runs never overlap). The `...String` forms take a property placeholder, `${name:default}`, so both delays are settings.

```
List<String> discover(String format)
```

The events asked for, newest first.

```
private static String label(String format)
```

The format's name for the logs.

## EventFetchRepository.java

`src/main/java/com/pkreference/backend/ingest/EventFetchRepository.java`

```
@Repository public class EventFetchRepository
```

The event_fetch table: what discovery has asked for, and when the fetcher got it. It's the pipeline's memory between polls: without it every poll would ask for every listed event again.

`@Repository` marks a class that talks to the database. Besides making it a bean, Spring turns the JDBC driver's exceptions into its own `DataAccessException` family here. `NamedParameterJdbcTemplate` runs SQL with named placeholders ( `:id` ) filled from a map, never by pasting values into the SQL text, so a value can't change the query (no SQL injection).

**MEMBERS**

```
public EventFetchRepository(NamedParameterJdbcTemplate jdbc)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
public Map<String, EventFetch> find(Collection<String> ids)
```

The fetch state of each of these events that has one, by event id. One query, with `in (:ids)` expanded to the list.

```
public void markRequested(Tournament t, Instant at)
```

Records that discovery asked for the event at `at` : an update, or an insert the first time. "Update, and insert if nothing was updated" is an upsert written portably, since H2 (the tests' database) and Postgres spell the one-statement form differently.

```
public void markFetched(Tournament t, Instant at, boolean standingsFinal)
```

Records that the fetcher got the event at `at` , and whether its standings are final.

```
private static Instant instant(ResultSet rs, int column) throws SQLException
```

A `timestamp with time zone` column as an Instant, or null.

```
private static OffsetDateTime timestamp(Instant instant)
```

An Instant as the type the JDBC drivers store in a `timestamp with time zone` column.

## EventFetcher.java

`src/main/java/com/pkreference/backend/ingest/EventFetcher.java`

```
@Component public class EventFetcher
```

tournaments.discovered -> the event's details, standings and pairings -> standings.fetched and pairings.fetched. Three Limitless requests an event, through the throttle. A failure throws, so the record retries and then goes to the DLT. Same consumer group as the standings fetcher it replaces, so a running backend carries on where it was.

This is the pipeline's slow worker. Under the throttle a request can wait minutes for its turn, which is why application.yml gives consumers few records a poll and a long `max.poll.interval.ms` : a consumer that goes too long between polls is assumed dead, and its records are handed out again.

**MEMBERS**

```
public EventFetcher(LimitlessClient client, EventFetchRepository fetches, KafkaTemplate<String, Object> kafka,
LimitlessProperties props, Clock clock)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@KafkaListener(topics = Topics.TOURNAMENTS_DISCOVERED, groupId = "standings-fetcher") public void
onDiscovered(TournamentDiscovered event)
```

Called by Spring Kafka for each record on tournaments.discovered. `@KafkaListener` makes Spring start a consumer thread that polls the topic, turns each record's JSON back into a `TournamentDiscovered` and calls this method with it.

`groupId` names the consumer group. Kafka remembers, per group, how far each partition has been read (the "offset"), so after a restart this listener resumes where it stopped. The offset moves on only when this method returns normally: if it throws, the record is retried (KafkaErrorHandling).

The order matters: publish both results, waiting for the broker to confirm each, and only then mark the event fetched. If anything fails part-way, nothing is marked, and the retry does the whole event again; the stores downstream accept a repeat.

## FetchPolicy.java

`src/main/java/com/pkreference/backend/ingest/FetchPolicy.java`

```
public final class FetchPolicy
```

Which listed events to fetch: the app's rules (TeamCorpusStore). Events of at least minPlayers that have started, within the lookback. Each is fetched once its standings are final (fetched settleAfter after it started) and again every refetchAfter until then.

Pure functions of their arguments: no Spring, no database, no clock of their own ("now" is passed in). That's deliberate: the rules are the part most worth testing, and FetchPolicyTest can check every branch with plain values.

**MEMBERS**

```
public record EventFetch(String eventId, Instant requestedAt, Instant fetchedAt, boolean standingsFinal)
```

What's been asked for and fetched of one event (EventFetchRepository).

> ```
> boolean isPending()
> ```
>
> Asked for, and not fetched since.

```
public static boolean shouldRequest(Tournament t, EventFetch state, Instant now, LimitlessProperties p)
```

Whether discovery should ask for this event now. `state` is what event_fetch says of it, null when it has never been asked for. In order: too small, not started or too old: no. Never asked: yes. Already final: no. Asked and still waiting: only if the request looks lost. Fetched but not final: only once refetchAfter has passed.

```
public static boolean isFinal(Tournament t, Instant fetchedAt, Duration settleAfter)
```

Standings fetched settleAfter after the event started won't change.

```
public static Instant startOf(Tournament t)
```

The event's start, or null when Limitless's date doesn't read.

## LimitlessClient.java

`src/main/java/com/pkreference/backend/ingest/LimitlessClient.java`

```
@Component public class LimitlessClient
```

Thin client for the Limitless play API, same endpoints as the app's LimitlessAPI actor.

`@Component` : Spring makes one of these at start-up and hands it to EventDiscovery and EventFetcher, whose constructors ask for it. `RestClient` is Spring's HTTP client; each call below is "GET this path, turn the JSON answer into these records".

Limitless allows 50 requests in 5 minutes without a key, so every request goes through a RequestThrottle first: discovery and the fetcher never need to think about the limit.

**MEMBERS**

```
@Autowired public LimitlessClient(RestClient.Builder builder, LimitlessProperties props)
```

The constructor Spring uses ( `@Autowired` picks it, since there are two): the real clock and a real sleep. `RestClient.Builder` is injected fresh for each class that asks, so configuring it here affects only this client.

```
LimitlessClient(RestClient.Builder builder, LimitlessProperties props, RequestThrottle throttle)
```

For tests, which pass a throttle on a fake clock. An interceptor wraps every request this client sends: wait for a turn, send, then tell the throttle what Limitless's `ratelimit` header said was left.

```
public List<Tournament> tournaments(String format, int page)
```

One page of the game's tournaments, newest first: all formats when format is null.

```
public TournamentDetails details(String tournamentId)
```

An event's organizer, platform and phases (Swiss rounds, top cut).

```
public List<Standing> standings(String tournamentId)
```

An event's standings, with each player's team when the event published them. `ParameterizedTypeReference` is how a generic type (`List<Standing>`) is named at run time, where Java has erased it: without it Jackson would build a list of plain maps.

```
public List<Pairing> pairings(String tournamentId)
```

The event's matches; none for an event Limitless has no pairings for (HTTP 404).

```
private <T> T withRetry(java.util.function.Supplier<T> call)
```

Honors 429 + Retry-After (capped at 30s), like LimitlessAPI.swift.

```
private static long retryAfterSeconds(HttpClientErrorException e)
```

The Retry-After header's seconds, kept between 1 and 30; 5 when it's missing or isn't a number.

### RequestThrottle.java

`src/main/java/com/pkreference/backend/ingest/RequestThrottle.java`

```
public class RequestThrottle
```

Keeps requests under a limit of `limit` in each `window`, as Limitless's keyless API allows (50 in 5 minutes): a sliding window of the requests sent, plus what Limitless says is left. The app shares the address when it runs on the same machine, so its requests count too, and only Limitless's `ratelimit` header sees them.

**MEMBERS**

```
public interface Sleeper
```

Sleeps, or in tests moves a clock on.

```
void sleep(Duration duration) throws InterruptedException
```

```
public RequestThrottle(int limit, Duration window, Clock clock, Sleeper sleeper)
```

`clock` and `sleeper` are passed in rather than being the system's, so a test can run hours of throttling in a millisecond: its fake clock moves when the fake sleeper is called.

```
public void acquire() throws InterruptedException
```

Waits until a request may go, and counts it.

```
public synchronized void observe(String header)
```

Reads a response's `ratelimit` header, such as `"50-in-5min"; r=0; t=212`: with nothing left, no request goes until the reset.

```
private static Integer number(Matcher m)
```

The first number the pattern captures, or null when the header doesn't have it.

## store: from Kafka to Postgres

The second half: the listeners that read the fetched topics and the repository that writes events, teams and matches.

### PairingResult.java

`src/main/java/com/pkreference/backend/store/PairingResult.java`

```
public enum PairingResult
```

How a Limitless match ended.

**MEMBERS**

```
P1, P2, TIE, DOUBLE_LOSS, BYE, NO_SHOW, UNKNOWN
```

The enum's values.

```
public boolean wasPlayed()
```

Whether both players played it: a match to count for or against them.

```
public static PairingResult of(Pairing p)
```

Reads Limitless's `winner` field, which holds a username, "0" (a tie) or "-1" (both lost). With no second player it's a bye when player 1 is the winner, otherwise a no-show. Storing the result as one word means every query can say `result in ('P1', 'P2', ...)` instead of repeating these rules in SQL.

## TeamStore.java

`src/main/java/com/pkreference/backend/store/TeamStore.java`

```
@Repository public class TeamStore
```

Every event's teams and matches, as fetched: the tables Phase 2's insights read. Each member gets its species key the app's way (SpeciesVocabularies). Each fetch of an event replaces its rows, so reading the topics again from the start rebuilds them, and a fetch older than the one stored is ignored.

The tables (db/migration/V2 to V4): `event` and its `event_phase` s, each player's `team` with its `team_member` s and their `team_member_move` s, every match in `pairing` , and `store_version` , a single counter.

Why "replace, don't update": a Kafka consumer can be given the same record twice (after a crash, a rebalance, or on purpose when the group's offsets are reset to rebuild the tables). Writing so that a repeat ends in the same rows, called idempotence, makes all of those safe.

`JdbcTemplate` is Spring's plain-SQL helper: `update` for one statement, `batchUpdate` for the same statement over many rows in one round trip, `query` to read. Values go in as `?` parameters, never into the SQL text.

### MEMBERS

```
public TeamStore(JdbcTemplate jdbc, SpeciesVocabularies species)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@Transactional public void putStandings(StandingsFetched fetched)
```

Stores one event's standings and teams, replacing what was stored for it.

`@Transactional` : Spring opens a database transaction before the method and commits it when the method returns; if it throws, everything it did is rolled back. So a reader never sees an event with its old rows deleted and its new ones half written, and a failure part-way leaves the previous copy in place for the retry.

```
@Transactional public void putPairings(PairingsFetched fetched)
```

Stores one event's matches, replacing what was stored for it, with each match's result worked out once (PairingResult). In its own transaction, as putStandings is. `pairing_fetch` remembers when the event's pairings were fetched, since an event can have none.

```
private void bumpVersion()
```

Committed with the write, so readers see a new version exactly when they see new rows.

```
private boolean isStale(Instant fetchedAt, String storedQuery, String id)
```

Whether a fetch is older than the one already stored. Records from before fetchedAt existed never are.

```
static Instant parseDate(String date)
```

Limitless's date text as an Instant, or null when it's missing or doesn't read.

```
private static OffsetDateTime timestamp(Instant instant)
```

An Instant as the type the JDBC drivers store in a `timestamp with time zone` column.

```
private static String cut(String s, int max)
```

Hand-typed fields are trimmed to their column, rather than failing the whole event.

## TeamStoreConsumer.java

`src/main/java/com/pkreference/backend/store/TeamStoreConsumer.java`

```
@Component public class TeamStoreConsumer
```

standings.fetched and pairings.fetched -> the team store. Its own consumer groups: resetting them to the start rebuilds the tables from the topics.

The same topics are also read by UsageConsumer, under a different `groupId`. That's the point of consumer groups: each group gets every record and keeps its own place, so two independent readers of one topic don't take records from each other.

This class is deliberately thin. The listener methods only hand the record to TeamStore, which holds the logic and the transaction and can be tested without Kafka.

**MEMBERS**

```
public TeamStoreConsumer(TeamStore store)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@KafkaListener(topics = Topics.STANDINGS_FETCHED, groupId = "team-store-standings") public void
onStandings(StandingsFetched event)
```

One event's standings. If the store throws, the record is retried and then sent to standings.fetched.DLT.

```
@KafkaListener(topics = Topics.PAIRINGS_FETCHED, groupId = "team-store-pairings") public void onPairings(PairingsFetched
event)
```

One event's matches. They can arrive before or after its standings: the tables don't depend on the order.

## standardize: one spelling for every name

Players type their teams by hand. These classes turn what they typed into canonical item, ability and move names, and into the app's species keys. Ported from the app, and checked against it by golden files.

### NameDictionary.java

`src/main/java/com/pkreference/backend/standardize/NameDictionary.java`

```
final class NameDictionary
```

Canonical spellings for one kind of name, with exact (normalized) and typo-tolerant lookup.

**MEMBERS**

```
NameDictionary(Collection<String> names)
```

Indexes each name by its key; the first spelling given for a key is the one kept.

```
static String key(String raw)
```

Lowercase letters and digits only: "Fake-out", "FAKE OUT" and "Fake Out" all become "fakeout".

```
String exact(String raw)
```

The name whose key is `raw`'s, or null: no near-misses, for IDs that weren't typed by hand.

```
String resolve(String raw, NameDictionary fallback)
```

Exact match here, then exact in `fallback`, then a unique near-miss in this dictionary and then in `fallback`; otherwise the input title-cased, so unknown names still merge across casing.

```
private String nearest(String k)
```

A single candidate within a small edit distance sharing the first letter. A tie returns null instead of guessing. Short names need an exact match.

```
private static int maxDistance(int length)
```

How many edits a near miss may be from a real name: none under 5 letters, 1 under 10, otherwise 2.

```
static int distance(String a, String b, int cap)
```

Optimal string alignment distance (edits, plus swapping two adjacent letters), capped.

```
static String titleCase(String raw)
```

"life ORB" -> "Life Orb"; capitalizes after spaces and hyphens ("never-melt ice" -> "Never-Melt Ice").

```
static final class Builder
```

Collects names from several files before the dictionary is made.

```
void add(String name)
```

Adds one name; blanks are ignored.

```
void addAll(Builder other)
```

Adds every name another builder holds.

```
NameDictionary build()
```

The dictionary of everything added.

## NameStandardizer.java

`src/main/java/com/pkreference/backend/standardize/NameStandardizer.java`

```
@Component public class NameStandardizer
```

Canonicalizes the free-text names players type into decklists (items, abilities, moves, natures, Tera types), so "MIRACLE SEED", "Miracle seed" and "miracle seed" count as one.

Item spellings also come from `reference/champions-items.json`, a snapshot of Serebii's Champions item page (every Champions item, not per regulation).

Names come from the repo-root regulation files (`champions-m-*.json` and their learnsets) plus `showdown-champions-data.json`, copied onto the classpath under `regulations/` at build time. Those lists are used to fix spelling and casing only, never to drop a value: decklists come from formats with no regulation file (fan formats, older or newer regulations), so a name that matches nothing is kept, title-cased.

### MEMBERS

```
private record Names(NameDictionary items, NameDictionary abilities, NameDictionary moves)
```

Names for one regulation, keyed by regulation id ("m-c"); the union is the fallback.

```
public NameStandardizer(ObjectMapper mapper)
```

Loads every name list once, at start-up. `ObjectMapper` is Jackson's JSON reader, a bean Spring Boot provides.

```
public String item(String format, String raw)
```

Blank and "none"-style values become `#NO_ITEM`; null stays null (no item given).

```
public String ability(String format, String raw)
```

An ability's canonical name; null for null or blank.

```
public String move(String format, String raw)
```

A move's canonical name; null for null or blank.

```
public String itemById(String format, String id)
```

A Showdown ID from Smogon's stats ("choicescarf") as the item's name ("Choice Scarf"); the ID itself when no item has it. Smogon's "nothing" is `#NO_ITEM`. Exact matches only: an ID is never misspelled, so a near-miss would be another item.

```
public String abilityById(String format, String id)
```

An ability's Showdown ID as its name; the ID itself when nothing has it.

```
public String moveById(String format, String id)
```

A move's Showdown ID as its name; the ID itself when nothing has it.

```
private String byId(String format, String id, java.util.function.Function<Names, NameDictionary> pick)
```

An exact lookup in the format's own lists, then in every regulation's.

```
public String nature(String raw)
```

A nature's canonical name. Natures and Tera types are the same in every regulation.

```
public String tera(String raw)
```

A Tera type's canonical name.

```
private String resolve(String format, String raw, java.util.function.Function<Names, NameDictionary> pick)
```

Looks a name up in the format's own lists, then in every regulation's.

```
private static JsonNode read(ObjectMapper mapper, Resource r) throws IOException
```

Parses one bundled JSON file, closing it afterwards.

## SpeciesNames.java

`src/main/java/com/pkreference/backend/standardize/SpeciesNames.java`

```
public final class SpeciesNames
```

The app's name helpers, ported rule for rule so species keys match it: toID (ShowdownCalc.swift), LimitlessSpeciesResolver.words (LimitlessTeamImport.swift) and TeamQueryParser.tokens. The golden file golden/species-identity.json checks the result against the app.

### FIELDS AND CONSTANTS

```
private static final Map<String, String> WORD_SUBSTITUTIONS
```

Region adjectives, and Limitless's "f"/"m" for gender forms.

```
private static final Map<String, String> FORM_SUFFIXES
```

A Showdown-style form suffix after a hyphen: "arcanine-h", "indeedee-f".

### MEMBERS

```
public static String toId(String text)
```

Lowercases and keeps only [a-z0-9], as Showdown's toID.

```
public static List<String> words(String name)
```

Lowercased ASCII words, accents folded, region adjectives and gender marks normalized.

```
public static List<String> tokens(String text)
```

The query parser's words, with a form suffix after a hyphen spelled out ("arcanine-h").

```
private static String fold(String s)
```

Accents removed and lowercased, as Swift's case- and diacritic-insensitive folding.

```
private static boolean isAsciiLetterOrDigit(char c)
```

a to z and 0 to 9 only: the text has been lowercased and had its accents removed by now.

## SpeciesVocabularies.java

`src/main/java/com/pkreference/backend/standardize/SpeciesVocabularies.java`

```
@Component public class SpeciesVocabularies
```

Each bundled regulation's SpeciesVocabulary, by Limitless format ("M-C" is champions-m-c.json). Maven bundles the app's files into regulations/, so a new regulation needs nothing here.

### MEMBERS

```
public record SpeciesKey(String key, String megaStone)
```

A member's species key and, when its item is one of its Mega Stones, the stone's ID.

```
public SpeciesVocabularies()
```

Reads the bundled files once, at start-up. They're on the classpath under `regulations/` because pom.xml copies them there from the repository's root, where the app keeps them.

```
public SpeciesKey identify(String format, String name, String slug, String item)
```

A member's species key the app's way. Outside a bundled regulation (a Scarlet and Violet format, a fan format) it's the slug, or the name when there's no slug, as a Showdown ID.

```
public Set<String> formats()
```

The bundled regulations' Limitless format ids, lowercased: "m-c".

```
public SpeciesVocabulary vocabulary(String format)
```

A bundled regulation's vocabulary, or null.

```
private static JsonNode read(ObjectMapper mapper, Resource resource) throws IOException
```

Parses one bundled JSON file, closing it afterwards.

### SpeciesVocabulary.java

`src/main/java/com/pkreference/backend/standardize/SpeciesVocabulary.java`

```
public final class SpeciesVocabulary
```

One regulation's species, as the app's TeamSearchVocabulary knows them: what species a Limitless member is (with the form words the regulation lists) and which Mega its held stone makes. Built from the same files: champions-<reg>.json, its learnsets, and team_search_vocab.json's form noise.

#### FIELDS AND CONSTANTS

```
private final Map<String, String> megaStones = new HashMap<>()
```

Stone ID to the species ID it's for.

#### MEMBERS

```
public record Identity(String speciesId, Set<String> formWords)
```

A member's species: "arcanine", or "arcanine:hisui" with its form words sorted.

```
public String key()
```

The species key the whole system uses: the ID alone, or the ID, a colon and its form words in alphabetical order.

```
private record Species(String id, String name, List<String> words, Set<String> formWords)
```

One species: its ID, name, the name's words, and the form words its regulation lists for it.

```
public SpeciesVocabulary(JsonNode regulation, JsonNode learnsets, JsonNode vocabulary)
```

Builds the lookups from the three files, already parsed. `JsonNode` is Jackson's tree form of JSON, read here by path ( `regulation.path("mega_stones")` ) because only a few fields of large files are wanted.

```
public Identity identity(String name, String slug)
```

A Limitless member's species, from its slug ("arcanine-hisui") when there is one, otherwise its display name ("Hisuian Arcanine"). A species outside the regulation keeps its slug or name as its ID.

```
public record ShowdownSpecies(Identity identity, boolean mega)
```

A Smogon or Showdown entry: its species, and whether the entry is the Mega.

```
public ShowdownSpecies showdown(String name)
```

A Smogon or Showdown species name, as the app's TeamSearchVocabulary.term(showdownName:) reads it: "Charizard-Mega-Y" is Charizard's Mega, "Arcanine-Hisui" is arcanine:hisui, "Indeedee-F" is indeedee:female. Null for a species outside the regulation.

```
public String megaStone(String heldItem, String speciesId)
```

The stone ID when the held item is one of the species' own Mega Stones, else null.

```
private Identity match(List<String> words, boolean prefixOnly)
```

The longest species whose words appear: at the start for a slug, anywhere for a name.

## insight: the tournament API

The read path for tournament data: SQL in the repository, counting in the service, HTTP in the controller, and the records that define the answers.

### MetaController.java

`src/main/java/com/pkreference/backend/insight/MetaController.java`

```
@RestController @RequestMapping("/v1") public class MetaController
```

API v1: read-only, the same for everyone, and cached. Responses may be reused for 15 minutes, and carry a weak ETag, so a client that has the current one gets a 304 with no body. Swagger UI at /swagger-ui.html shows it, from these annotations.

A controller is the web's edge of the application: it turns an HTTP request into a method call and the returned object into JSON. It holds no logic of its own. Each method here checks its parameters, asks MetaService, and wraps the answer with caching headers.

The annotations on a method are of two kinds. Spring's route the request: `@GetMapping("/formats/{format` /pokemon")} says which path and verb, `@PathVariable` takes a piece of the path, `@RequestParam` a query parameter. springdoc's ( `@Tag` , `@Operation` , `@Parameter` ) only describe the endpoint for Swagger UI and change nothing about how it runs.

#### FIELDS AND CONSTANTS

```
static final CacheControl CACHE = CacheControl.maxAge(Duration.ofMinutes(15)).cachePublic()
```

`Cache-Control: max-age=900, public` : any cache may reuse an answer for 15 minutes. With API keys configured, ApiAuthFilter rewrites `public` to `private` on the way out.

#### MEMBERS

```
public MetaController(MetaService meta)
```

Spring passes in the one MetaService bean.

```
@GetMapping("/formats") public ResponseEntity<Formats> formats()
```

GET /v1/formats. `ResponseEntity` is a response with its status and headers settable, where returning the body alone would only allow a plain 200. The app's Test Connection button calls this one.

```
@GetMapping("/formats/{format}/pokemon") public ResponseEntity<PokemonList> pokemon(String format, String window)
```

GET /v1/formats/{format}/pokemon?window=30d: the Meta tab's list. A window that isn't one is a 400 (MetaWindow.parse throws).

```
@GetMapping("/formats/{format}/pokemon/{key}") public ResponseEntity<PokemonDetail> pokemon(String format, String key,
String window)
```

GET /v1/formats/{format}/pokemon/{key}: one Pokémon's page. Java tells this method from the one above by its parameters.

```
@GetMapping("/formats/{format}/cores") public ResponseEntity<Cores> cores(String format, String window)
```

GET /v1/formats/{format}/cores: pairs and trios chosen together.

```
@GetMapping("/formats/{format}/archetypes") public ResponseEntity<Archetypes> archetypes(String format, String window)
```

GET /v1/formats/{format}/archetypes: the Meta tab's Teams to Beat.

```
@GetMapping("/formats/{format}/archetypes/{id}") public ResponseEntity<ArchetypeDetail> archetype(String format, String
id, String window)
```

GET /v1/formats/{format}/archetypes/{id}: one archetype's page.

```
@GetMapping("/formats/{format}/events") public ResponseEntity<Events> events(String format, int limit)
```

GET /v1/formats/{format}/events?limit=10: the Meta tab's Recent Events. A limit outside 1 to 50 is a 400.

```
@ExceptionHandler(IllegalArgumentException.class) ResponseEntity<Map<String, String>> badRequest(IllegalArgumentException
e)
```

`@ExceptionHandler` : when a method of this controller throws IllegalArgumentException (a window that isn't one, a limit out of range), Spring calls this instead of answering 500, and the client gets a 400 with the reason as JSON.

```
@ExceptionHandler(NoSuchElementException.class) ResponseEntity<Map<String, String>> notFound(NoSuchElementException e)
```

The same for NoSuchElementException, which MetaService throws for a Pokémon or archetype the window doesn't have: a 404.

## MetaRepository.java

`src/main/java/com/pkreference/backend/insight/MetaRepository.java`

```
@Repository public class MetaRepository
```

Reads the team store for the insights. Plain rows only: the counting is MetaService's, in Java, so the SQL stays the same on H2 and Postgres.

The pattern in every method: SQL in a text block ( """ ), values passed as `?` parameters, and a lambda that turns each row of the result into a record. A lambda taking `(rs, n)` is a RowMapper, returning one object a row, and `jdbc.query` returns the list of them; one taking only `rs` is a RowCallbackHandler, used where the rows are gathered into a map or a set instead.

Most queries take a window, `[from, to)` : events dated from `from` up to but not including `to` . `from` is null for the whole regulation, and then the `event_date >= ?` condition and its parameter are both left off.

#### FIELDS AND CONSTANTS

```
private static final RowMapper<PlacingRow> PLACING
```

Shared by placings and winners, which select the same columns.

#### MEMBERS

```
public MetaRepository(JdbcTemplate jdbc)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
public record FormatRow(String format, int events, int teams, Instant firstEvent, Instant lastEvent, Instant lastFetched)
```

One format's stored events.

```
public record MemberRow(String eventId, String player, Instant eventDate, int slot, String speciesKey, String megaStone,
String item, String ability, String nature)
```

One team member, with its event's date.

```
public String team()
```

"eventId|player": how a team is identified everywhere in the insights.

```
public List<FormatRow> formats()
```

Each stored format's events, teams with a published team, date range and newest fetch. The `left join` keeps events that have no such teams, counting them as events with 0 teams.

```
public long version()
```

The team store's version, which each write bumps: what results are memoized on.

```
public List<MemberRow> members(String format, Instant from, Instant to)
```

Every member of a team with a published team, at events of the format in [from, to).

```
public List<String[]> moves(String format, String speciesKey, Instant from, Instant to)
```

The moves of one species' members in [from, to), as "event|player|slot" and the move.

```
public record TopCut(Set<String> teams, Set<String> bracketEvents)
```

Teams that made a top cut ("event|player"): who played in a bracket phase (phase > 1). Events with a bracket are in bracketEvents.

```
public TopCut topCut(String format, Instant from, Instant to)
```

Who played in a bracket phase, and which events had one, in [from, to).

```
public record PairingRow(String eventId, String player1, String player2, String result)
```

A match both players played: result is P1, P2, TIE or DOUBLE_LOSS (PairingResult).

```
public String team1()
```

Player 1's team, as "eventId|player".

```
public String team2()
```

Player 2's team, as "eventId|player".

```
public List<PairingRow> pairings(String format, Instant from, Instant to)
```

Every played match at events of the format in [from, to): byes and no-shows aren't games.

```
public record EventRow(String id, String name, Instant date, int players, boolean standingsFinal)
```

One stored event.

```
public List<EventRow> events(String format, int limit)
```

The format's newest stored events.

```
public Map<String, Integer> topCutPlayers(List<String> eventIds)
```

How many players played in each event's top cut (a bracket phase); absent when it had none.

```
public record PlacingRow(String eventId, String player, String name, Integer placing, int wins, int losses, int ties,
String eventName, Instant eventDate, int players)
```

One player's entry, with its event: what an example team or a winner needs.

```
public String team()
```

"eventId|player".

```
public List<PlacingRow> placings(String format, Instant from, Instant to)
```

Every entry at events of the format in [from, to).

```
public List<PlacingRow> winners(List<String> eventIds)
```

First place at each of these events.

```
public record TeamMemberRow(String team, int slot, String speciesKey, String name, String item, String ability, String
nature, List<String> moves)
```

One member of a chosen team, with its moves in order.

```
public List<TeamMemberRow> teamMembers(Set<String> teams)
```

The members of these teams ("event|player"), in slot order.

```
private static String placeholders(int n)
```

"?, ?, ?" for an `in (...)` list of n values: the values still go in as parameters.

```
private static Instant instant(ResultSet rs, int column) throws SQLException
```

A `timestamp with time zone` column as an Instant, or null.

```
private static OffsetDateTime timestamp(Instant instant)
```

An Instant as the type the JDBC drivers compare with a `timestamp with time zone` column.

## MetaResponses.java

`src/main/java/com/pkreference/backend/insight/MetaResponses.java`

```
public final class MetaResponses
```

The /v1 responses. Shares are 0 to 1, as the app's are. Every response says what it's from: the window it covers, when it was worked out, and how many events and teams.

Records, written to JSON by Jackson field for field: these declarations are the API's contract with the app, whose MetaAPI.swift mirrors them. A boxed type ( `Double` , `Integer` ) marks a field that can be null in the JSON. `@Schema` only feeds Swagger UI's descriptions.

### MEMBERS

```
public record Sample(int events, int teams, int topCutTeams)
```

What the numbers rest on.

```
public record Format(String format, int events, int teams, Instant firstEvent, Instant lastEvent, Instant lastFetched)
```

One stored format, in GET /v1/formats.

```
public record Formats(Instant generatedAt, List<Format> formats)
```

GET /v1/formats.

```
public record WinRecord(int wins, int losses, int ties, int matches, Double winRate, Double winRateLow, Double
winRateHigh)
```

Match results for teams with it, from Limitless's pairings. Mirror matches (both teams have it) don't count; a tie counts as half a win, and a double loss as a loss. winRate and its 95% range (Wilson) are null under 30 matches.

```
public record PokemonUsage(String key, int teams, double usage, int topCutTeams, Double topCutUsage, Double trend,
WinRecord record)
```

```
public record PokemonList(String format, String window, Instant from, Instant to, Instant generatedAt, Sample sample,
List<PokemonUsage> pokemon)
```

GET /v1/formats/{f}/pokemon.

```
public record Share(String value, int count, double share)
```

A value, and how many of the Pokémon's members (or, for teammates, its teams) had it.

```
public record PokemonSet(String item, String ability, String nature, List<String> moves, int count, double share)
```

A whole set, counted as one: item, ability, nature and its moves.

```
public record WeekUsage(LocalDate week, int teams, int withPokemon, double usage)
```

Usage in one week (Monday to Sunday, UTC) of the window.

```
public record PokemonDetail(String format, String key, String window, Instant from, Instant to, Instant generatedAt,
Sample sample, PokemonUsage usage, List<Share> items, List<Share> abilities, List<Share> natures, List<Share> moves,
List<Share> megaStones, List<Share> teammates, List<PokemonSet> sets, List<WeekUsage> weekly)
```

GET /v1/formats/{f}/pokemon/{key}.

```
public record Core(List<String> members, int teams, double share, double lift)
```

Pokémon that are on teams together.

```
public record Cores(String format, String window, Instant from, Instant to, Instant generatedAt, Sample sample,
List<Core> pairs, List<Core> trios)
```

GET /v1/formats/{f}/cores.

```
public record Matchup(String against, WinRecord record)
```

One archetype's record against another, from the matches between them.

```
public record Archetype(String id, String name, List<String> core, int teams, double usage, int topCutTeams, Double topCutUsage, WinRecord record, List<Matchup> matchups)
```

Teams built around a core of four Pokémon. Cores are the most common sets of four that differ from each more common core by at least two Pokémon; each team belongs to the most common core it contains, and teams with none are "other".

```
public record Archetypes(String format, String window, Instant from, Instant to, Instant generatedAt, Sample sample, int other, List<Archetype> archetypes)
```

GET /v1/formats/{f}/archetypes.

```
public record ExampleMember(String key, String name, String item, String ability, String nature, List<String> moves)
```

```
public record ExampleTeam(String eventId, String eventName, Instant eventDate, int players, String player, Integer placing, int wins, int losses, int ties, List<ExampleMember> members)
```

One team in an archetype, with where it placed.

```
public record ArchetypeDetail(String format, String window, Instant from, Instant to, Instant generatedAt, Sample sample, Archetype archetype, List<ExampleTeam> examples)
```

GET /v1/formats/{f}/archetypes/{id}.

```
public record EventWinner(String player, int wins, int losses, int ties, List<String> team)
```

An event's winner: first place.

```
public record EventSummary(String id, String name, Instant date, int players, boolean standingsFinal, Integer topCutPlayers, EventWinner winner)
```

One event, in GET /v1/formats/{f}/events.

```
public record Events(String format, Instant generatedAt, List<EventSummary> events)
```

GET /v1/formats/{f}/events.

## MetaService.java

`src/main/java/com/pkreference/backend/insight/MetaService.java`

```
@Service public class MetaService
```

The insights, worked out from the team store when asked: usage in a window, its share of top-cut teams, its trend, and each Pokémon's sets, items, moves and teammates. Results are kept until the store changes (or the hour does, which moves the windows), so a burst of requests is one computation. Definitions are BackendIntegration-PLAN.md §4.2's.

The design choice: there are no "usage" tables. Each number is computed from the stored teams at the moment it's asked for, in memory, and then remembered. At a few thousand teams a format that takes milliseconds, and it means a change to a definition is a code change with nothing to migrate or backfill.

How a request flows: MetaController calls a public method here; it reads rows through MetaRepository; `Teams.of` indexes them both ways (each team's Pokémon, each Pokémon's teams); the counting happens over those two maps; the result is rounded into a MetaResponses record. Every public method does this inside `memoize`.

A team is identified by "eventId|player" throughout, and a Pokémon by its species key ("arcanine:hisui").

### FIELDS AND CONSTANTS

```
static final Duration TREND_PERIOD = Duration.ofDays(14)
```

Trend: the last 14 days against the 14 before.

```
static final int TREND_MIN_TEAMS = 50
```

Fewer teams than this in either period, and there's no trend.

```
static final int MIN_MATCHES = 30
```

Fewer matches than this, and there's no win rate.

```
static final int MIN_CORE_TEAMS = 4
```

A core or archetype needs at least this many teams, and at least ARCHETYPE_SHARE of them.

```
static final int EXAMPLE_TEAMS = 5
```

An archetype page's example teams.

**MEMBERS**

```
public MetaService(MetaRepository repository, NameStandardizer names, Clock clock)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
public Formats formats()
```

GET /v1/formats: each stored format's counts and dates.

```
public PokemonList pokemon(String format, MetaWindow window)
```

GET /v1/formats/{f}/pokemon: every Pokémon on a team in the window, most used first, each with its usage, top-cut usage, trend and win record. Four reads (members, top-cut teams, the trend's two fortnights, pairings), then counting.

```
public PokemonDetail pokemon(String format, String key, MetaWindow window)
```

One Pokémon's page. Throws NoSuchElementException when no team in the window had it.

```
public Cores cores(String format, MetaWindow window)
```

The pairs and trios most often on teams together, with how much more often than chance.

```
private static List<Core> topCores(Teams teams, int size)
```

The `size`-Pokémon combinations on the most teams. Lift is the share of teams with all of them over the product of their separate usages: what chance would give if they were chosen independently. Above 1, they're picked together more than chance explains.

```
public Archetypes archetypes(String format, MetaWindow window)
```

Archetypes: cores of four, the most common first, each differing from every more common core by at least two Pokémon (sharing at most two), with at least MIN_CORE_TEAMS teams and ARCHETYPE_SHARE of them. Each team belongs to the first core it contains. Each is named by its two most-used members, and by more of its core when an archetype with more teams has that name.

```
static List<Archetype> named(List<Archetype> archetypes)
```

Names in order (most teams first): the two most-used members, or as many more as it takes to differ from every name before it.

```
public ArchetypeDetail archetype(String format, String id, MetaWindow window)
```

One archetype, with its best-placed teams. Throws NoSuchElementException for an id not in the window.

```
public Events events(String format, int limit)
```

The format's newest stored events, with their top cuts and winners.

```
private record Grouping(Instant from, Teams teams, Set<String> topCutTeams, List<List<String>> cores, Map<String, String>
archetypeOf, Map<String, Set<String>> members)
```

Teams grouped into archetypes, before anything is counted: what the list and each page share.

```
private Grouping grouping(String format, MetaWindow window, Instant now)
```

Sorts the window's teams into archetypes. Counts every four-Pokémon combination, keeps those on enough teams, most common first, and drops any that shares three or more Pokémon with one already kept. Then each team goes to the first kept core it contains. Memoized by itself, because the archetype list and each archetype's page both start from it.

```
static List<List<String>> combinations(Set<String> keys, int size)
```

Every sorted combination of `size` keys.

```
private static void combine(List<String> keys, int size, int start, List<String> current, List<List<String>> out)
```

The recursion behind combinations: extend `current` with each later key in turn, and record it when it's full.

```
private Map<String, Double> trends(String format, Instant now)
```

Usage in the last 14 days less the 14 before, for each key; absent when either period is thin.

```
private static PokemonUsage usage(String key, Set<String> withIt, int teams, Set<String> topCutTeams, Double trend, int[]
record)
```

One Pokémon's row: its teams, its share of all teams and of top-cut teams (null when there were none), its trend and its record.

```
static Map<String, int[]> records(Teams teams, List<PairingRow> pairings)
```

Each Pokémon's wins, losses and ties ([w, l, t]) in matches between two teams with published lists, leaving out mirror matches: a Pokémon on both teams says nothing about itself.

```
private static int outcome(String result, boolean player1)
```

0 win, 1 loss, 2 tie, for player 1 or player 2. A double loss is a loss for both.

```
private static void tally(Map<String, int[]> records, Set<String> mine, Set<String> theirs, int outcome)
```

Adds the outcome to every Pokémon on `mine` that isn't also on `theirs`.

```
static WinRecord winRecord(int[] r)
```

Win rate (a tie is half a win) with its 95% Wilson range; both null under MIN_MATCHES.

```
private static List<WeekUsage> weekly(Teams teams, Set<String> withIt)
```

Usage in each week of the window, weeks starting on Monday (UTC), oldest first: the page's chart.

```
private static List<Share> shares(List<MemberRow> members, Function<MemberRow, String> value, int total)
```

Counts one field (item, ability, nature, Mega Stone) over a Pokémon's members and returns the commonest, as shares of `total`.

```
private static List<Share> top(Map<String, Integer> counts, int total)
```

The TOP_VALUES biggest counts, as shares of `total`; equal counts in name order, so the answer doesn't change between calls.

```
void forget()
```

Drops every memoized result (tests, whose rolled-back writes reuse versions).

```
private <T> T memoize(String key, Instant now, Supplier<T> compute)
```

Not computeIfAbsent: one result can be worked out from another (an archetype page from the grouping), and a ConcurrentHashMap can't be changed from inside its own computeIfAbsent. Two requests at once may both compute a result; the answers are the same.

```
private static Set<String> intersect(Set<String> a, Set<String> b)
```

The elements of `a` that are also in `b`.

```
private static String memberId(MemberRow m)
```

One team member, as "event|player|slot": what MetaRepository.moves keys its rows by.

```
private static String blankToNull(Object o)
```

"" back to null: sets are keyed with "" for a missing part, since List.of takes no nulls.

```
private static double ratio(int part, int whole)
```

`part / whole`, rounded; 0 when there's nothing to divide by.

```
private static double round(double x)
```

To four decimal places: shares are sent as 0.4819, not 0.48192771084337.

```
record Teams(Map<String, Set<String>> keysByTeam, Map<String, Set<String>> teamsByKey, Map<String, Instant> dateByTeam,
Set<String> eventIds)
```

Teams with a published team: each one's Pokémon (once each) and its event's date.

> ```
> static Teams of(List<MemberRow> rows)
> ```
>
> Indexes member rows both ways. Members with no species key (stored before keys were added) are skipped.
>
> ```
> int count()
> ```
>
> How many teams have a published team.
>
> ```
> int events()
> ```
>
> How many events those teams are from.

## MetaWebConfig.java

`src/main/java/com/pkreference/backend/insight/MetaWebConfig.java`

```
@Configuration public class MetaWebConfig
```

ETags for /v1, worked out from each response's body: a matching If-None-Match gets a 304. Weak ETags (W/"..."), because Tomcat doesn't gzip a response with a strong one, and a gzipped body is the same content in another encoding.

### MEMBERS

```
@Bean FilterRegistrationBean<ShallowEtagHeaderFilter> v1Etags()
```

Registers Spring's ETag filter for every path under /v1. A servlet filter sees each request before the controller and each response after it. This one holds the response back, hashes its body into an `ETag` header, and compares that with the request's `If-None-Match`: when they're equal, the client already has this answer, and it gets a 304 Not Modified with no body. The server still does the work; what's saved is the transfer.

`FilterRegistrationBean` is how a filter is given its URL patterns and order; a filter declared as a plain bean would apply to every path.

### MetaWindow.java

`src/main/java/com/pkreference/backend/insight/MetaWindow.java`

```
public enum MetaWindow
```

The time a response covers: the last 14 or 30 days, or the whole regulation (every stored event).

**MEMBERS**

```
DAYS_14("14d", Duration.ofDays(14)), DAYS_30("30d", Duration.ofDays(30)), REGULATION("regulation", null)
```

The enum's values.

```
MetaWindow(String id, Duration length)
```

An enum constant's own fields: its spelling in the API and its length, null for the whole regulation.

```
public String id()
```

The window as the API spells it: "14d", "30d" or "regulation".

```
public Instant from(Instant now)
```

The window's start, or null for the whole regulation.

```
public static MetaWindow parse(String text)
```

"14d", "30d" or "regulation"; anything else is an error the caller reports.

## corpus: stored events in Limitless's own shape

Two endpoints that let the app build its Team Search data from the server with the code it uses for Limitless.

### CorpusController.java

`src/main/java/com/pkreference/backend/corpus/CorpusController.java`

```
@RestController @RequestMapping("/v1") public class CorpusController
```

The stored events in Limitless's shapes, for the app's Team Search corpus: the same two calls it makes to Limitless (a page of a format's tournaments, and one tournament's standings), answered from the server, so devices stop crawling Limitless each.

**MEMBERS**

```
public CorpusController(CorpusRepository corpus)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@GetMapping("/formats/{format}/tournaments") public ResponseEntity<List<Tournament>> tournaments(String format, int page,
int limit)
```

GET /v1/formats/{format}/tournaments?page=1&limit=50. `defaultValue` makes a parameter optional. A limit over 100 is cut to 100 rather than refused, as a page size usually is.

```
@GetMapping("/tournaments/{id}/standings") public ResponseEntity<List<Standing>> standings(String id)
```

GET /v1/tournaments/{id}/standings: what the app's Team Search asks for each event it lists.

```
@ExceptionHandler(IllegalArgumentException.class) ResponseEntity<Map<String, String>> badRequest(IllegalArgumentException
e)
```

A page or limit under 1: 400, with the reason as JSON.

```
@ExceptionHandler(NoSuchElementException.class) ResponseEntity<Map<String, String>> notFound(NoSuchElementException e)
```

An event that isn't stored: 404, with the reason as JSON.

### CorpusRepository.java

`src/main/java/com/pkreference/backend/corpus/CorpusRepository.java`

```
@Repository public class CorpusRepository
```

The team store read back in Limitless's own shapes (Events.Tournament and Events.Standing), so the app's Team Search corpus can be built from the server with the code that builds it from Limitless.

#### FIELDS AND CONSTANTS

```
static final DateTimeFormatter LIMITLESS_DATE = DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss.SSS'Z'")
```

Limitless's date format: "2026-10-05T18:00:00.000Z".

#### MEMBERS

```
public CorpusRepository(JdbcTemplate jdbc)
```

Spring passes in the JdbcTemplate it built on the configured data source.

```
public List<Tournament> tournaments(String format, int page, int limit)
```

One page (from 1) of the format's events, newest first, as Limitless lists them.

```
public List<Standing> standings(String eventId)
```

The event's standings with their teams, or null when the event isn't stored.

```
private static String orEmpty(String s)
```

Limitless always sends these as text; a stored null goes back as "".

## ladder: Smogon's monthly stats

A second, smaller pipeline with the same parts as the first: a client, a scheduled discovery, a reducer, a topic, a store, a service and a controller.

### LadderController.java

`src/main/java/com/pkreference/backend/ladder/LadderController.java`

```
@RestController @RequestMapping("/v1/ladder") public class LadderController
```

API v1's ladder: read-only and cached as the rest of /v1 is (MetaController).

#### FIELDS AND CONSTANTS

```
static final CacheControl CACHE = CacheControl.maxAge(Duration.ofMinutes(15)).cachePublic()
```

`Cache-Control: max-age=900, public`, as MetaController's: 15 minutes.

```
private static final Pattern MONTH = Pattern.compile("^\\d{4}-(0[1-9]|1[0-2])$")
```

A month as the API takes it: four digits, a hyphen, 01 to 12.

#### MEMBERS

```
public LadderController(LadderService ladder)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@GetMapping public ResponseEntity<Index> index()
```

GET /v1/ladder: what the app reads first, to find the format id for a regulation's doubles or singles ladder.

```
@GetMapping("/{format}") public ResponseEntity<Stats> stats(String format, Integer rating, String month)
```

GET /v1/ladder/{format}?rating=1760&month=2026-09. `required = false` with a boxed type (`Integer`) lets a parameter be left out, arriving as null for the service to default. A rating that isn't a number never reaches this method: Spring answers 400 itself.

```
@ExceptionHandler(IllegalArgumentException.class) ResponseEntity<Map<String, String>> badRequest(IllegalArgumentException e)
```

A month that isn't one: 400, with the reason as JSON.

```
@ExceptionHandler(NoSuchElementException.class) ResponseEntity<Map<String, String>> notFound(NoSuchElementException e)
```

A format, month or rating that isn't stored: 404, with the reason as JSON.

### LadderDiscovery.java

`src/main/java/com/pkreference/backend/ladder/LadderDiscovery.java`

```
@Component @ConditionalOnProperty(name = "pkref.smogon.ingest-enabled", havingValue = "true", matchIfMissing = true)
```

```
public class LadderDiscovery
```

Looks through Smogon's newest months for Champions ladder stats not stored yet, and fetches, reduces and publishes each on ladder.stats. A month appears on the 1st of the next and never changes, so each file is fetched once; a longer lookback, set once, is the backfill.

#### FIELDS AND CONSTANTS

```
private static final Pattern FILE = Pattern.compile("^(.+)-(\\d+)\\.json$")
```

"gen9championsvgc2026regmc-1760.json": the format, then the rating.

#### MEMBERS

```
public LadderDiscovery(SmogonClient client, LadderReducer reducer, LadderStore store, SpeciesVocabularies vocabularies,
KafkaTemplate<String, Object> kafka, SmogonProperties props, Clock clock)
```

Constructor injection: each parameter is a bean Spring supplies.

```
@Scheduled(initialDelayString = "${pkref.smogon.initial-delay:20s}", fixedDelayString = "${pkref.smogon.poll-
interval:12h}") public void poll()
```

Run by Spring's scheduler 20 seconds after start-up, then 12 hours after each run ends. The scheduler has two threads (application.yml), so this and EventDiscovery's poll don't wait on each other.

```
List<String> discover()
```

The records published, as their topic keys, newest month first.

## LadderFormat.java

src/main/java/com/pkreference/backend/ladder/LadderFormat.java

```
public record LadderFormat(String id, String regulation, String mode, int bestOf)
```

One of Smogon's Champions ladder formats, read from its id: "gen9championsvgc2026regmc" is M-C doubles, with "bo3" on the end best of three; "gen9championsbssregmc" is M-C singles (Battle Stadium Singles). The doubles id has the year in it, so formats are found by this pattern in Smogon's listing, never built from a regulation.

#### FIELDS AND CONSTANTS

```
private static final Pattern ID = Pattern.compile("^gen\\d+champions(vgc\\d{4}|bss)reg([a-z0-9]+?)(bo3)?$")
```

"gen", digits, "champions", then "vgc" and a year or "bss", then "reg" and the regulation's letters, then optionally "bo3". The regulation's group is reluctant ( `+?` ) so that a trailing "bo3" is matched as best-of-three rather than as part of the regulation.

#### MEMBERS

```
public static LadderFormat parse(String id, Collection<String> regulations)
```

Null when the id isn't a Champions doubles or singles ladder, or its regulation isn't one of `regulations` (Limitless ids such as "m-c", in any case).

## LadderReducer.java

src/main/java/com/pkreference/backend/ladder/LadderReducer.java

```
@Component public class LadderReducer
```

Cuts one of Smogon's chaos files, and its leads file, down to a Ladder.Stats (BackendIntegration-PHASE5.md §3). The app's SmogonLadder.reduce does the same, rule for rule, and both must produce golden/ladder-fixture.json from the fixture, so a change here is a change there, a new `#REDUCTION` and the golden file recorded again.

To get the same bits in Java and Swift: sums are taken in the order of the keys, as bytes; equal counts are ordered by key, as bytes; shares are rounded half to even at five places, and usage and lead shares at nine. Smogon writes some numbers to more digits than a double holds ("0.0032668000000000003"), and JSON parsers don't all read those to the same double (Foundation's JSONSerialization lands a few units in the last place away), so nothing goes out unrounded.

#### FIELDS AND CONSTANTS

```
public static final int REDUCTION = 1
```

The version of this rule. Stats stored at an older one are fetched and reduced again.

```
static final double FLOOR = 0.005
```

Abilities, items, moves and natures on under 0.5% of the Pokémon's teams are left out.

```
static final Comparator<String> BYTES
```

Strings by their UTF-8 bytes: the same order in Java and in Swift, whatever the letters.

MEMBERS

```
private record Count(String id, double count)
```

One entry of a chaos file's objects: an id (an item, a move, a spread, a teammate) and its weighted count.

```
public LadderReducer(ObjectMapper mapper, SpeciesVocabularies vocabularies, NameStandardizer names)
```

Spring passes in its JSON mapper and the two name services; all three are beans.

```
public Stats reduce(InputStream chaos, String leads, LadderFormat format, String month, int rating) throws IOException
```

Reads the chaos file as a stream, one Pokémon at a time: the biggest files are over 7 MB, and one Pokémon's part is a small tree. `leads` is the leads file's text, or null.

```
private Pokemon pokemon(String name, JsonNode entry, LadderFormat format, SpeciesVocabulary vocabulary, Double leadShare)
```

One Pokémon's entry, or null when it has no usage or no weight to take shares of.

```
private static List<Count> counts(JsonNode object)
```

An object of counts, in the order of its keys.

```
private static double sum(List<Count> counts)
```

The counts added up in list order, which is key order: adding doubles in another order can give a different last bit.

```
private static List<Count> ranked(List<Count> counts)
```

Biggest first; equal counts by key.

```
private static List<Share> shares(List<Count> counts, double weight, int limit, double floor, Function<String, String>
name)
```

The `limit` biggest as shares of `weight`, leaving out those under `floor`, those of nothing, and those `name` has no name for.

```
private static List<Count> natures(List<Count> spreads)
```

The spreads' counts added up by nature: a spread is "Adamant:2/32/0/0/0/32".

```
static Map<String, Double> leadShares(String leads)
```

Each Pokémon's share of all leads, from the leads file's table: `| 1 | Sneasler | 6.59479% | 204684 | 6.271% |`.

```
static double round(double share)
```

To five places, half to even: Swift's rounded(.toNearestOrEven).

```
static double roundUsage(double usage)
```

Usage and lead shares, which Smogon publishes to seven places: to nine, half to even.

## LadderResponses.java

`src/main/java/com/pkreference/backend/ladder/LadderResponses.java`

```
public final class LadderResponses
```

The /v1/ladder responses. Shares are 0 to 1.

MEMBERS

```
public record Entry(String format, String regulation, String mode, int bestOf, String month, int rating, long battles,
Instant fetchedAt)
```

One stored month of a format at a rating.

```
public record Index(List<Entry> stats)
```

```
public record RatingUsage(int rating, double usage)
```

One Pokémon's usage at one rating cutoff.

```
public record Pokemon(String name, String key, String megaStone, double usage, Double trend, List<RatingUsage>
usageByRating, Double leadShare, List<Share> abilities, List<Share> items, List<Share> moves, List<Share> natures,
List<Share> spreads, List<Share> teammates)
```

One Pokémon's month, as Ladder.Pokemon has it, with what the other stored files add.

```
public record Stats(int reduction, String format, String regulation, String mode, int bestOf, String month, int rating,
long battles, Instant fetchedAt, String previousMonth, List<Integer> ratings, List<Pokemon> pokemon)
```

GET /v1/ladder/{format}: a stored record's fields, when it was fetched, and what the other stored files add.

## LadderService.java

src/main/java/com/pkreference/backend/ladder/LadderService.java

```
@Service public class LadderService
```

The ladder stats, as /v1/ladder serves them: a stored record, with what the other stored files add to it (each Pokémon's change since the month before, and its usage at the other ratings). Nothing is kept in memory: a record is one row, read when it's asked for.

### MEMBERS

```
public LadderService(LadderStore store)
```

Spring passes in the store.

```
public Index index()
```

GET /v1/ladder: every stored format, month and rating, without the Pokémon.

```
public Stats stats(String format, Integer rating, String month)
```

A format's month at a rating: 1760 and the newest stored month when they aren't given.

## LadderStore.java

src/main/java/com/pkreference/backend/ladder/LadderStore.java

```
@Repository public class LadderStore
```

The ladder_stats table: each reduced month of Smogon's stats, as ladder.stats delivered it. A newer record of a format, month and rating replaces the stored one.

The record is kept whole, as JSON text in one column, rather than spread over tables: it's written once, read whole, and never queried inside. A second column holds just each Pokémon's usage, so the service can compare months and ratings without parsing megabytes.

### FIELDS AND CONSTANTS

```
private static final TypeReference<LinkedHashMap<String, Double>> USAGE = new TypeReference<>()
```

Names the generic type the usage column's JSON is read into: a map from Pokémon to usage.

### MEMBERS

```
public record Entry(String format, String regulation, String mode, int bestOf, String month, int rating, long battles,
int reduction, Instant fetchedAt)
```

One stored record, without its Pokémon.

```
public record Stored(Stats stats, Instant fetchedAt)
```

A stored record, and when it was fetched.

```
public LadderStore(JdbcTemplate jdbc, ObjectMapper mapper)
```

Spring passes in its JdbcTemplate and JSON mapper.

```
@Transactional public boolean put(StatsFetched fetched)
```

Stores a record, unless the one stored was made by a newer rule, or by the same rule from a newer fetch. False when it was left out.

```
public boolean has(String format, String month, int rating, int reduction)
```

Whether a record is stored, made by this rule or a newer one.

```
public List<Entry> index()
```

Everything stored: by format, then newest month first, then rating.

```
public Optional<String> newestMonth(String format, int rating)
```

The newest month stored for a format at a rating.

```
public Optional<Stored> find(String format, String month, int rating)
```

One stored record, parsed back from its JSON, or empty when that month isn't stored at that rating.

```
public Map<Integer, Map<String, Double>> usage(String format, String month)
```

Each Pokémon's usage in a format's month, by rating and then by its name.

```
private String write(Object value)
```

A value as JSON text. A failure here would be a bug, not bad input, so it's rethrown unchecked.

```
private <T> T read(String json, Class<T> type)
```

JSON text from the table back into a record.

## LadderStoreConsumer.java

`src/main/java/com/pkreference/backend/ladder/LadderStoreConsumer.java`

```
@Component public class LadderStoreConsumer
```

ladder.stats -> the ladder_stats table. Its own consumer group: resetting it to the start rebuilds the table from the topic.

### MEMBERS

```
public LadderStoreConsumer(LadderStore store)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@KafkaListener(topics = Topics.LADDER_STATS, groupId = "ladder-store") public void onStats(StatsFetched event)
```

Called for each record on ladder.stats, already turned from JSON back into a `StatsFetched`. The store decides whether it's newer than what it has.

## SmogonClient.java

`src/main/java/com/pkreference/backend/ladder/SmogonClient.java`

```
@Component public class SmogonClient
```

Smogon's monthly stats (https://www.smogon.com/stats/): plain directory listings and files, the ones the app's SmogonLiveFetcher reads. They're a courtesy, not an API, so requests go one at a time with a gap between them.

### FIELDS AND CONSTANTS

```
private static final Pattern FILE = Pattern.compile("^[a-z0-9]+-\\d+\\.(json|txt)$")
```

A stats file's name: only these go into a request's path.

### MEMBERS

```
public interface BodyReader<T>
```

Reads a response's body while it downloads.

```
T read(InputStream body) throws IOException
```

```
@Autowired public SmogonClient(RestClient.Builder builder, SmogonProperties props)
```

The constructor Spring uses. It gives the client timeouts, and a throttle of one request per `requestGap` (2 seconds) on the real clock.

```
SmogonClient(RestClient.Builder builder, SmogonProperties props, RequestThrottle throttle)
```

For tests, which pass a throttle on a fake clock and a builder bound to a mock server. Every request waits for its turn in the throttle first, and names the project in its User-Agent.

```
private static SimpleClientHttpRequestFactory timeouts()
```

A stuck connection would hold the scheduler's thread for good.

```
public List<String> months()
```

The months with stats, newest first: "2026-09".

```
public List<String> chaosFiles(String month)
```

The files in a month's chaos directory: "gen9championsvgc2026regmc-1760.json".

```
public <T> T chaos(String month, String file, BodyReader<T> reader)
```

A chaos file, read as it downloads: the biggest are over 7 MB.

```
public String leads(String month, String file)
```

A leads file ("gen9championsvgc2026regmc-1760.txt"), or null when Smogon has none (HTTP 404).

```
private List<String> links(String path)
```

The link targets on a directory listing.

```
private static String text(byte[] body)
```

A response's bytes as text. Read as bytes and decoded here, so the encoding is UTF-8 whatever the server's headers say.

```
private static String checked(Pattern pattern, String value)
```

Returns `value` only when it matches `pattern`. Month and file names come from Smogon's own listing page and go into the next request's path, so anything that isn't plainly a month or a stats file's name ("../", a full URL) is refused here rather than requested.

## security: signing in

A servlet filter that checks every request, a service that issues and verifies tokens, the endpoint that trades a key for a token, and the configuration that wires them.

### ApiAuthFilter.java

`src/main/java/com/pkreference/backend/security/ApiAuthFilter.java`

```
public class ApiAuthFilter extends OncePerRequestFilter
```

Who gets in, checked before anything else sees the request. With API keys configured (ApiAuthProperties), every request carries `Authorization: Bearer <credential>`:

- `POST /v1/auth/token` takes an **API key**, and AuthController answers with an access token. The key is checked by its SHA-256 hash, in constant time.
- Everything else takes an **access token** (TokenService), not the key: the long-lived key is then sent once every few minutes rather than with every request, and a token that leaks stops working by itself.

Anything else gets a 401 that says why in `code`, so a client knows whether to trade its key again (`token_expired`, `invalid_token`) or stop (`invalid_key`). Neither credential is ever read from the URL, where proxies and logs keep it, or logged.

Swagger UI and the OpenAPI document are let in without either: they describe the API, as the repository does, and hold no data.

Answers that say they may be reused publicly (`Cache-Control: public`) say `private` instead, so a cache in front of the server can't hand a signed-in answer to a caller who isn't.

#### FIELDS AND CONSTANTS

```
public static final String KEY_ID = ApiAuthFilter.class.getName() + ".keyId"
```

The request attribute holding the id of the key a token request came with, for AuthController.

#### MEMBERS

```
public ApiAuthFilter(ApiAuthProperties properties, TokenService tokens)
```

Not a `@Component`: SecurityConfig creates it and registers it, so its order and paths are set in one place. The hashes are turned from hex text into bytes once, here, for the comparison in keyId.

```
@Override protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
throws ServletException, IOException
```

Runs for every request, before any controller. `chain.doFilter` passes the request on to the next filter and finally the controller; returning without calling it ends the request here.

In order: no keys configured, or a documentation path: let it through. No credential: 401. The token path: the credential must be an API key. Anything else: it must be a valid access token, and the response is wrapped so its caching is marked private.

```
private static String bearer(HttpServletRequest request)
```

What follows "Bearer " in the Authorization header, or null.

```
private static void refuse(HttpServletRequest request, HttpServletResponse response, String code, String message) throws
IOException
```

Answers 401 with a JSON body `{"error": ..., "code": ...}`, never cached. The log line names the path and the reason but not the credential.

```
private static String path(HttpServletRequest request)
```

The request's path within the application, as the client sent it.

```
static boolean isDocumentation(HttpServletRequest request)
```

Swagger UI's page and assets, and the OpenAPI document it reads. The path is the request's as sent; one that Tomcat would rewrite before routing it ("/swagger-ui/../v1/formats", encoded characters, ";" parameters, "//") never counts, so it can't reach data without signing in.

```
String keyId(String key)
```

The id of the configured key `key` is, or null when it isn't one. Every hash is compared, each in constant time, so how long this takes says nothing about which key came close.

```
static byte[] sha256(String key)
```

SHA-256 of the key's bytes. Every JVM has SHA-256, so the checked exception can't happen in practice.

```
private static final class PrivateCaching extends HttpServletResponseWrapper
```

Turns `Cache-Control: public` into `private`.

```
PrivateCaching(HttpServletResponse response)
```

Wraps the real response: every other call passes straight through to it.

```
@Override public void setHeader(String name, String value)
```

Controllers set Cache-Control through one of these two; both are rewritten.

```
@Override public void addHeader(String name, String value)
```

As setHeader: the other way a header can be written.

```
private static String privately(String name, String value)
```

The value with "public" replaced by "private", when the header is Cache-Control; otherwise as it is.

## ApiAuthProperties.java

`src/main/java/com/pkreference/backend/security/ApiAuthProperties.java`

```
@ConfigurationProperties(prefix = "pkref.api") public record ApiAuthProperties(List<String> keyHashes, Duration tokenLifetime, String tokenSecret)
```

Who may use the API, and how (README, "Signing in to the API").

`keyHashes` : the API keys the server accepts, as the SHA-256 hashes of the keys (hex), so the configuration never holds a key itself. None, and every request is let in, as on a server only this machine can reach. Set them with PKREF_API_KEY_HASHES (comma-separated); backend/scripts/new-api-key.sh makes a key and its hash.

`tokenLifetime` : how long an access token, which a key is traded for, is good for. 15 minutes unless set (PKREF_TOKEN_LIFETIME, such as "30m").

`tokenSecret` : what signs the tokens (PKREF_TOKEN_SECRET), at least 32 characters. Unset, the server makes a random one each time it starts: tokens from before a restart stop working, and the app trades its key for a new one. Set it only to keep tokens across restarts.

### FIELDS AND CONSTANTS

```
static final int MIN_SECRET_BYTES = 32
```

HS256 signs with a 256-bit key: a shorter secret is easier to guess than the signature is to forge.

```
public ApiAuthProperties
```

Checked when the record is built, which is at start-up: a mistake in the configuration (a key where a hash belongs, a secret that's too short) stops the server from starting, with a message saying what to fix, instead of leaving it running open or weak.

### MEMBERS

```
public static ApiAuthProperties ofKeys(List<String> keyHashes)
```

Only keys, with the default lifetime and a random secret.

```
public boolean required()
```

Whether signing in is on: true once at least one key hash is configured.

```
static String keyId(String hash)
```

A short name for a key, from its hash: what a token says it was traded for, and what the logs say, without either holding the key or its whole hash.

## AuthController.java

`src/main/java/com/pkreference/backend/security/AuthController.java`

```
@RestController @RequestMapping("/v1/auth") public class AuthController
```

Trades an API key for an access token. ApiAuthFilter has already checked the key by the time a request gets here, and says which key it was (ApiAuthFilter.KEY_ID); this only makes the token.

#### FIELDS AND CONSTANTS

```
static final String OPEN = "open"
```

With no keys configured the server lets everyone in; a token then names this in place of a key.

#### MEMBERS

```
public record TokenResponse(String token, String tokenType, long expiresIn, Instant expiresAt)
```

An access token, and how long it's good for.

```
public AuthController(TokenService tokens, ApiAuthProperties properties)
```

Spring passes in the token service SecurityConfig made, and the settings.

```
@PostMapping("/token") public ResponseEntity<?> token(HttpServletRequest request)
```

POST /v1/auth/token. Takes no body: the API key is in the Authorization header, and ApiAuthFilter has already checked it and left the key's id in a request attribute. Answers with the token, its lifetime in seconds and its expiry time.

## SecurityConfig.java

`src/main/java/com/pkreference/backend/security/SecurityConfig.java`

```
@Configuration public class SecurityConfig
```

Signing in to the API, put together: the tokens, and the check ahead of every other filter and on every path but Swagger UI's (/v1 and the older /api/usage).

`@Configuration` marks a class whose `@Bean` methods Spring calls once each at start-up, keeping what they return as beans. It's the alternative to `@Component` for objects that need arguments or set-up a bare constructor call can't express, as the filter's registration below does.

This project doesn't use Spring Security. One filter and one small token service do what it needs, and can be read from top to bottom.

#### FIELDS AND CONSTANTS

```
static final String API_KEY = "apiKey"
```

Swagger UI's names for the two credentials.

#### MEMBERS

```
@Bean TokenService tokenService(ApiAuthProperties properties, ObjectProvider<Clock> clock)
```

One for the whole server: its secret is what makes a token made here verify here.

`ObjectProvider<Clock>` asks for a bean that may not exist: the application defines a Clock, but a test that builds only the web layer may not, and then the system clock is used.

```
@Bean FilterRegistrationBean<ApiAuthFilter> apiAuth(ApiAuthProperties properties, TokenService tokens)
```

Registers the filter on every path ("/*") and puts it first: `HIGHEST_PRECEDENCE` means it runs before the ETag filter and everything else, so nothing does any work for a request that will be refused.

```
@Bean OpenAPI openApi(ApiAuthProperties properties)
```

Swagger UI's Authorize button, when signing in is required: the API key, which only the token request takes, and the access token, which everything else does. Authorize with the key, run POST /v1/auth/token, then Authorize with the token it returns.

## TokenService.java

`src/main/java/com/pkreference/backend/security/TokenService.java`

```
public class TokenService
```

The API's access tokens: JSON Web Tokens (JWTs), which an API key is traded for and every other request then carries.

A token is three parts joined by dots: a header (how it was signed), claims (who it's for and when it runs out) and a signature over both. The first two are only base64, readable by anyone, so nothing secret goes in them; the signature is what makes them trustworthy. It's HMAC-SHA256 ("HS256") with a secret only this server has, since this server both signs and verifies.

Verifying is strict about the things JWT libraries have been fooled by: the algorithm must be HS256 (not "none", and not whatever the token says), the signature must match, the token must not have run out, and it must have been made by this server, for this API, from a key that's still configured, so removing a key's hash also ends its tokens.

#### MEMBERS

```
public enum Refusal
```

Why a token was refused.

```
public static final class InvalidTokenException extends Exception
```

Thrown by verify, carrying why. A checked exception, so the filter has to decide what to answer.

```
InvalidTokenException(Refusal refusal)
```

The exception's message is the refusal's name, for the logs of anything that prints it.

```
public Refusal refusal()
```

Why the token was refused.

```
public record Issued(String token, Instant expiresAt, long expiresInSeconds)
```

A token just made, and when it stops working.

```
public TokenService(ApiAuthProperties properties, Clock clock)
```

Made by SecurityConfig. Uses the configured secret, or 32 random bytes when none is set. Keeps the short ids of the configured keys, to check each token's subject against.

```
private static byte[] randomSecret()
```

32 bytes from the operating system's secure random source, new each start.

```
public Issued issue(String keyId)
```

A token for the holder of the key named `keyId`, good for the configured lifetime.

```
public String verify(String token) throws InvalidTokenException
```

The id of the key `token` was traded for, when the token is good; otherwise why it isn't.

## usage: the first version's counters

All-time counts kept with JPA. Superseded by the team store and the insights, still running, and the plainest example here of consume, transact, produce.

### ProcessedTournament.java

`src/main/java/com/pkreference/backend/usage/ProcessedTournament.java`

```
@Entity public class ProcessedTournament
```

Marker written in the same transaction as the counters, so replays never double-count.

**MEMBERS**

```
protected ProcessedTournament()
```

For Hibernate, which builds entities through a no-argument constructor.

```
public ProcessedTournament(String id)
```

Marks the event with this id as counted.

```
public String getId()
```

Returns the field.

### ProcessedTournamentRepository.java

`src/main/java/com/pkreference/backend/usage/ProcessedTournamentRepository.java`

```
public interface ProcessedTournamentRepository extends JpaRepository<ProcessedTournament, String>
```

Which events have been counted. Spring Data writes the implementation; `existsById` and `save` are all that's used.

### UsageAggregationService.java

`src/main/java/com/pkreference/backend/usage/UsageAggregationService.java`

```
@Service public class UsageAggregationService
```

Turns one event's standings into counter changes: how many teams ran each Pokémon, and for each Pokémon how many ran each item, ability, move, Tera type and nature.

`@Service` is `@Component` under another name: it makes the class a bean, and says to the reader that it holds business logic rather than talking to the web or the database. The database work goes through the two repositories it's given.

**MEMBERS**

```
public UsageAggregationService(UsageCounterRepository counters, ProcessedTournamentRepository processed, NameStandardizer names)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@Transactional public List<UsageUpdated> apply(StandingsFetched event)
```

Folds one tournament into the counters and records it as processed in the same transaction. Replaying the same tournament is a no-op (returns an empty list).

```
private static UsageCounter newCounter(String key)
```

A zeroed counter for a key, "format|category|species|value"; the value may itself contain "|", hence the limit of 4.

```
private static void bump(Map<String, Long> deltas, String format, String category, String species, String value)
```

Adds one to a counter's pending change. Nothing to count when the value is missing.

```
private static String speciesKey(TeamMember m)
```

The Pokémon's key for these counters: Limitless's slug, or its name lowercased. (The team store uses the app's species key instead.)

```
private static String lower(String s)
```

Lowercased the same way in every locale.

## UsageConsumer.java

src/main/java/com/pkreference/backend/usage/UsageConsumer.java

```
@Component public class UsageConsumer
```

standings.fetched -> counters -> pokemon.usage. Runs single-threaded (default concurrency 1) because counters are shared across tournaments of one format.

This and the classes beside it are the first version of the backend: all-time counts served at /api/usage. The team store and MetaService replaced them for the app; they still run, and are the simplest whole example here of consume, transact, produce.

### MEMBERS

```
public UsageConsumer(UsageAggregationService service, KafkaTemplate<String, Object> kafka)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
@KafkaListener(topics = Topics.STANDINGS_FETCHED, groupId = "usage-aggregator") public void onStandings(StandingsFetched event)
```

Counts one event, then publishes each changed Pokémon's new total. Its own consumer group ("usage-aggregator"), so it reads standings.fetched independently of the team store.

## UsageCounter.java

src/main/java/com/pkreference/backend/usage/UsageCounter.java

```
@Entity @Table(indexes = @Index(columnList = "fmt, category, species")) public class UsageCounter
```

One counter row. category is TEAMS (species = "*", the format's team total), SPECIES (teams running the species), or ITEM / ABILITY / MOVE / TERA / NATURE (value = what they ran).

A JPA entity: `@Entity` tells Hibernate that objects of this class are rows of a table (named after the class, usage_counter), `@Id` marks the primary key and `@Column` names a column that differs from its field. Hibernate needs the no-argument constructor to build objects from rows. The table itself is made by Flyway (V1); with `ddl-auto: validate`, Hibernate only checks at start-up that the two agree.

This and ProcessedTournament are the only JPA entities. Everything newer uses JdbcTemplate and plain SQL, which reads more directly for bulk replaces and reporting queries.

### MEMBERS

```
public UsageCounter(String format, String category, String species, String value)
```

A new counter at zero, with its key made from the four parts.

```
public static String key(String format, String category, String species, String value)
```

The row's primary key: the four parts joined, so a counter can be fetched by what it counts.

```
public void add(long n)
```

Adds to the count. Hibernate notices the change and writes it when the transaction commits.

```
public String getId()
```

Accessors, for Hibernate and the controller. Entities are ordinary mutable classes, not records: Hibernate fills and tracks their fields.

```
public String getFormat()
```

Returns the field.

```
public String getCategory()
```

Returns the field.

```
public String getSpecies()
```

Returns the field.

```
public String getValue()
```

Returns the field.

```
public long getCount()
```

Returns the field.

### UsageCounterRepository.java

`src/main/java/com/pkreference/backend/usage/UsageCounterRepository.java`

```
public interface UsageCounterRepository extends JpaRepository<UsageCounter, String>
```

The counters' queries. There is no implementation in this project: Spring Data JPA writes one at start-up. `JpaRepository<UsageCounter, String>` brings `findById`, `findAllById`, `saveAll` and the rest, and each method below is a query spelled as its name: `findByFormatAndCategoryOrderByCountDesc` becomes `where format = ? and category = ? order by count desc`.

**MEMBERS**

```
List<UsageCounter> findByFormatAndCategoryOrderByCountDesc(String format, String category, Pageable page)
```

A format's counters of one category, biggest first; `page` limits how many.

```
List<UsageCounter> findByFormatAndSpeciesAndCategoryNotInOrderByCountDesc(String format, String species, List<String>
excluded)
```

Everything counted about one Pokémon in a format, leaving out the categories given.

## api: the first version's endpoint

Serves the counters at /api/usage.

### UsageController.java

`src/main/java/com/pkreference/backend/api/UsageController.java`

```
@RestController @RequestMapping("/api/usage") public class UsageController
```

The first version's API: all-time usage from the counters, at /api/usage. Nothing in the app calls it now (the app reads /v1); it's kept until the plan's Phase 7.

`@RestController` makes this a bean whose methods answer HTTP requests, with whatever they return written as JSON. `@RequestMapping("/api/usage")` is the path every method here starts from.

**MEMBERS**

```
public UsageController(UsageCounterRepository counters)
```

Constructor injection: Spring supplies each parameter from the beans it has made.

```
public record SpeciesUsage(String species, long teams, double usagePercent)
```

One row of the list: a Pokémon, the teams running it and that as a percentage of all teams.

```
public record SpeciesDetail(String species, long teams, double usagePercent, Map<String, List<ValueCount>> breakdown)
```

One Pokémon's page: its totals, then its counters grouped by category ("item", "move" and so on).

```
public record ValueCount(String value, long count)
```

A value (an item, a move) and how many of the Pokémon's sets had it.

```
@GetMapping public List<SpeciesUsage> top(String format, int limit)
```

GET /api/usage?format=M-C&limit=50: the most-used Pokémon. `@RequestParam` binds a query parameter to an argument; one without a default is required, and leaving it out is a 400. `limit` is kept between 1 and 500.

```
@GetMapping("/{species}") public SpeciesDetail detail(String species, String format)
```

GET /api/usage/{species}?format=M-C: one Pokémon's items, abilities, moves, Tera types and natures. `@PathVariable` binds the `{species}` part of the path. Throwing `ResponseStatusException` is the short way to answer with a status, here 404.

```
private long totalTeams(String format)
```

The format's team count, kept as a counter of its own (category TEAMS).

```
private static double percent(long n, long total)
```

A percentage to two places; 0 when there are no teams.

## The files that aren't Java

| File | What it is |
|---|---|
| `pom.xml` | The Maven build: the Spring Boot parent (which manages versions), the dependencies, and a resources section that copies the app's data files (champions-*.json, showdown-champions-data.json, team_search_vocab.json) from the repository into the jar under regulations/, so the server and the app read the same lists. |
| `src/main/resources/application.yml` | Every setting: the data source, JPA, the scheduler's pool, the Kafka producer and consumer, and the project's own blocks (pkref.limitless, pkref.smogon, pkref.api). Part 4.5 and Part 10.2 explain the values. |
| `src/main/resources/db/migration/V1__usage_counters.sql` | The first version's two tables: usage_counter and processed_tournament. |
| `src/main/resources/db/migration/V2__events_teams_pairings.sql` | event_fetch and the team store: event, event_phase, team, team_member, team_member_move, pairing, pairing_fetch. |
| `src/main/resources/db/migration/V3__team_member_species.sql` | Adds species_key and mega_stone to team_member, with an index on the key. |
| `src/main/resources/db/migration/V4__store_version.sql` | The one-row store_version table the insights' cache is keyed on. |
| `src/main/resources/db/migration/V5__ladder_stats.sql` | ladder_stats: one row per format, month and rating. |
| `src/main/resources/reference/champions-items.json` | A snapshot of every Champions item's name, for NameStandardizer. |
| `docker-compose.yml` | Local development: Kafka (one KRaft node), Kafka UI and Postgres, each published on 127.0.0.1 only, with named volumes so data survives a restart. |
| `docker-compose.server.yml` | Layered on top for a server: adds the API as a container, memory limits, restart policies, a health check for Postgres, and requires a real password and at least one API key hash. |
| `Dockerfile` | A two-stage build: Maven and a JDK build the jar; a JRE image runs it as a non-root user. |
| `scripts/new-api-key.sh` | Prints a new API key and its SHA-256 hash. The key goes into the app, the hash into the server's settings. |
| `scripts/backup.sh` | A compressed pg_dump of the database, kept 14 days, for a daily cron job. |
| `../.github/workflows/backend.yml` | CI: runs mvn verify on every push and pull request that touches the backend or the data files it bundles. |

# Part 9. The tests

`mvn verify` runs 137 tests in 30 classes. No test calls Limitless or Smogon: they read fixtures saved once from the real services (`src/test/resources/limitless`, with players' names replaced, and `src/test/resources/smogon`). GitHub Actions runs the suite on every change to the backend.

## 9.1 Five kinds of test

| Kind | How it's built | What it proves | Examples |
|---|---|---|---|
| **Plain unit tests** | Just `new` the class. No Spring. | Rules and arithmetic. | `FetchPolicyTest`, `RequestThrottleTest`, `LadderReducerTest`, `TokenServiceTest`, `PairingResultTest` |
| **Database slice** | `@JdbcTest`: only the data source, Flyway's migrations and the repositories named in `@Import`, on in-memory H2. | The SQL and the counting, on the real schema. | `TeamStoreTest`, `MetaServiceTest`, `LadderStoreTest`, `EventDiscoveryTest` |
| **The same on Postgres** | The same test class body, with Testcontainers starting Postgres 17 in Docker. Skipped where there's no Docker. | That the SQL is valid Postgres, which H2 doesn't guarantee. | `TeamStorePostgresTest`, `MetaServicePostgresTest`, `LadderStorePostgresTest`, `CorpusRepositoryPostgresTest` |
| **Web slice** | `@WebMvcTest`: the controllers and filters with a mocked service, driven through `MockMvc`. | Paths, parameters, status codes, headers, JSON shape, signing in. | `MetaControllerTest`, `LadderControllerTest`, `ApiAuthFilterTest` |
| **Pipeline** | `@SpringBootTest` with `@EmbeddedKafka`: the whole application and a real broker inside the test's JVM, with only the HTTP client mocked. | That a record published at one end arrives, stored, at the other. | `EventPipelineTest`, `UsagePipelineTest`, `LadderPipelineTest` |

## 9.2 Golden files

The app repeats three of the server's computations on the device, for when there is no server. To keep the two from drifting, both test suites check the same recorded files under `src/test/resources/golden`:

- `species-identity.json`: how team members' names become species keys (`SpeciesVocabularyGoldenTest`).
- `meta-fixture.json`: the insights for the fixture event (`MetaGoldenFileTest`).
- `ladder-fixture.json`: the ladder fixture reduced (`LadderGoldenFileTest`).

A change on either side fails a test until the other side matches.

## 9.3 Techniques worth naming

- **Injected clocks.** Time-dependent code takes a `Clock`; tests pass a fixed or movable one (`RequestThrottleTest.FakeClock`, `TokenServiceTest.TestClock`).
- **Mocking the edge only.** `LimitlessClient` and `SmogonClient` are mocked; everything between them and the database is real in the pipeline tests.
- **A real server for path tricks.** `ApiAuthTomcatTest` starts Tomcat and sends raw requests over a socket, because `MockMvc` doesn't normalize paths the way a server does.
- **Awaitility.** Pipeline tests wait for an asynchronous result with `await().untilAsserted(...)` instead of sleeping.

## 9.4 Every test class

| Class | Tests | What it covers |
|---|---|---|
| `LimitlessFixtures` | helpers | One real Limitless event (M-C, 83 players, 2026-10-05), as its API answered, with the players' names and usernames replaced (player-01 is first place) and the organizer renamed. |
| `corpus/CorpusRepositoryPostgresTest` | 2 | CorpusRepositoryTest on Postgres 17. |
| `corpus/CorpusRepositoryTest` | 2 | The store read back in Limitless's shapes is what Limitless sent: the contract that lets the app build its Team Search corpus from the server with its Limitless code. |
| `ingest/EventDiscoveryTest` | 4 | Discovery against a canned list, with the event_fetch table on the real migrations. |
| `ingest/EventPipelineTest` | 2 | tournaments.discovered -> the fetcher (Limitless mocked with the fixture event) -> both topics -> the team store and the usage counters, on an in-JVM broker. |

| | | |
|---|---|---|
| ingest/FetchPolicyTest | 3 | The app's rules: 16 players or more, final 48 hours after the start, refetched every 6 hours. |
| ingest/LimitlessClientTest | 1 | Every request goes through the throttle, which hears Limitless's own count. |
| ingest/RequestThrottleTest | 3 | |
| insight/MetaControllerTest | 3 | The /v1 web layer: JSON shape, caching headers, ETags, and errors. |
| insight/MetaGoldenFileTest | 1 | golden/meta-fixture.json: the insights MetaService works out from the fixture event, for the app's version worked out on the device (BackendIntegration-PHASE4.md §6) to match. |
| insight/MetaServicePostgresTest | 11 | MetaServiceTest's insights on Postgres 17, as docker-compose.yml runs it. |
| insight/MetaServiceTest | 11 | The insights over the real fixture event, on the migrations. |
| ladder/LadderControllerTest | 4 | /v1/ladder's web layer: JSON shape, caching headers, ETags, parameters and errors. |
| ladder/LadderDiscoveryTest | 6 | Discovery against a canned Smogon, with the ladder_stats table on the real migrations: what it fetches, what it leaves, and that a file is fetched once. |
| ladder/LadderFixtures | helpers | Smogon's real 2026-09 stats for M-C doubles at 1760, saved once and cut down to 27 Pokémon, each with its 25 biggest spreads and 30 biggest teammates; every count kept is as published. |
| ladder/LadderFormatTest | 2 | |
| ladder/LadderGoldenFileTest | 1 | golden/ladder-fixture.json: the fixture (LadderFixtures) as LadderReducer reduces it. |
| ladder/LadderPipelineTest | 1 | Discovery (Smogon mocked with the fixture) -> ladder.stats -> the ladder store -> what /v1/ladder serves, on an in-JVM broker: the record comes through the topic whole. |
| ladder/LadderReducerTest | 8 | The reducer's rules, on the fixture and on input made to be awkward. |
| ladder/LadderStorePostgresTest | 7 | The ladder store's tests on Postgres 17, as docker-compose.yml runs it. |
| ladder/LadderStoreTest | 7 | The ladder store's tests on H2. |
| ladder/LadderStoreTestBase | 7, run by its subclasses | The ladder's store and what the service makes of it, on the migrations. |
| ladder/LadderTestConfig | helpers | What the store's tests need that a @JdbcTest doesn't bring, and records to store. |
| ladder/SmogonClientTest | 4 | Smogon's listings and files, as its pages were on 2026-10-12, and the wait between requests. |
| security/ApiAuthFilterTest | 10 | Signing in, through the web layer: a key is traded for a token, the token opens the API until it runs out, and nothing else does. |
| security/ApiAuthTomcatTest | 3 | Signing in, in a real Tomcat, with raw requests: a key is traded for a token and the token opens /v1; and paths Tomcat rewrites before routing them (dot segments, encoded characters, ";"... |
| security/TokenServiceTest | 9 | The access tokens: what a good one holds, and every way a bad one is refused. |
| standardize/NameStandardizerTest | 10 | Runs against the real regulation files bundled from the repo root. |
| standardize/SpeciesVocabularyGoldenTest | 2 | Species keys match the app's. |
| store/PairingResultTest | 2 | |
| store/TeamStorePostgresTest | 5 | The store's tests, and the fetch table's, on Postgres 17 as docker-compose.yml runs it. |
| store/TeamStoreTest | 4 | The store's tests on H2. |
| store/TeamStoreTestBase | 4, run by its subclasses | The real fixture event through the store, on the migrations. |
| usage/Fixtures | helpers | |
| usage/UsageAggregationServiceTest | 5 | |
| usage/UsagePipelineTest | 4 | standings.fetched -> aggregator -> DB -> REST, against an in-JVM Kafka broker. |

# Part 10. Running it

## 10.1 Locally

```
cd backend
docker compose up -d          # Kafka (9092), Kafka UI (8081), Postgres (5432), all on 127.0.0.1
mvn spring-boot:run           # the server, on 127.0.0.1:8080
curl localhost:8080/v1/formats
open http://localhost:8080/swagger-ui.html
mvn verify                    # every test
```

On first start Flyway creates the tables, the topics are created, and within seconds discovery starts asking for the last 7 days' events and the ladder discovery fetches the two newest months.

## 10.2 The settings

| Environment variable | Default | Meaning |
|---|---|---|
| `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD` | local Postgres, `pkref` | Where the database is |
| `KAFKA_BOOTSTRAP_SERVERS` | `localhost:9092` | Where the broker is |
| `SERVER_ADDRESS` | `127.0.0.1` | Which network interface the API listens on |
| `LIMITLESS_FORMATS` | (all of the game's) | Formats to follow, e.g. `M-C,M-B` |
| `LIMITLESS_LOOKBACK` | `7d` | How far back each poll walks; longer once to backfill |
| `SMOGON_LOOKBACK_MONTHS` | `2` | How many recent months of ladder stats to keep up with |
| `SMOGON_INGEST_ENABLED` | `true` | Turn the Smogon poll off |
| `PKREF_API_KEY_HASHES` | (none: open) | SHA-256 hashes of accepted API keys |
| `PKREF_TOKEN_LIFETIME` | `15m` | How long an access token lasts |
| `PKREF_TOKEN_SECRET` | (random each start) | The token signing secret, 32 characters or more |

## 10.3 On a server

`docker-compose.server.yml` layers on top of the local file: it adds the API as a container built by the `Dockerfile` (two stages: build with Maven, run on a JRE as a non-root user), sets memory limits and restart policies, and requires a real Postgres password and at least one key hash. Everything still binds to `127.0.0.1`; other devices reach the API through Tailscale's HTTPS. `scripts/backup.sh` dumps Postgres daily; `scripts/new-api-key.sh` makes a key and its hash.

## 10.4 Rebuilding the tables from the topics

With the server stopped, reset a consumer group to the start of its topic and start the server again:

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka:19092 \
  --group team-store-standings --reset-offsets --to-earliest --all-topics --execute
```

The group re-reads every record, and because each write replaces what's there, the tables end up as if they had been filled for the first time with today's code.

# Part 11. Design decisions, and questions to expect

Short answers you can say aloud, each with the code that backs it up. Where the honest answer is "it's a limitation", that's what is written: an interviewer hears a trade-off understood, which is better than a defence.

## Architecture

### Why Kafka for a few hundred events a month?

For replay, not throughput. The fetched data stays in compacted topics for good, so any table can be rebuilt, with new rules, without crawling Limitless again. It also gives independent consumers and isolates the slow, rate-limited fetching from everything else. The cost is running a broker, kept to one KRaft node. A queue table in Postgres would handle the volume; it wouldn't give replay as naturally. (Part 4.3)

### Why are the insights computed on request instead of stored in tables?

At a few thousand teams a format, counting in memory takes milliseconds, and the result is cached until the data changes. Storing aggregates would mean migrations and backfills every time a definition changed. If a format grew to where this was slow, the plan is to move that insight to its own consumer with its own tables, rebuilt from the topics. ( `MetaService` )

### How do you know a cached answer isn't stale?

The cache key includes `store_version` , a counter that `TeamStore` increments inside the same transaction as every write. A reader sees the new version exactly when it can see the new rows. The key also includes the hour, because windows like "the last 14 days" move even when nothing is written. ( `MetaService.memoize` )

### What are the write path and read path, and why separate them?

Writes are driven by timers and Kafka records; reads by HTTP requests. They share only Postgres. So an outage of Limitless, Smogon or Kafka costs freshness, not availability, and a slow backfill never slows a request.

### Why split discovery from fetching?

Discovery is one cheap list call per page; fetching is three throttled calls per event and can run for hours. With a topic between them, a poll finishes quickly, a failed fetch retries alone, and pending work survives a restart. ( `EventDiscovery` , `EventFetcher` )

## Kafka

### What delivery guarantee do you have?

At-least-once consumption, with idempotent handlers, which gives exactly-once results. Producers use `acks=all` with idempotence and every send is awaited. I don't use Kafka transactions: the handlers write to Postgres, so the idempotence has to live there anyway. (Part 4.4)

### How is each consumer idempotent?

The team store deletes and re-inserts an event in one transaction and ignores a fetch older than the stored one. The ladder store replaces a row unless the stored one is newer. The counters, which can't be "replaced", record each processed event id in the same transaction as the counts.

### What happens to a record that keeps failing?

Three retries two seconds apart, then it's published to `<topic>.DLT` with the exception in its headers, and the consumer moves on. ( `KafkaErrorHandling` )

### What is compaction and why use it here?

The broker keeps at least the latest record per key instead of deleting by age. Keyed by event id, the topic holds the latest standings of every event ever fetched: a durable table I can re-read. An event fetched five times before its standings settled takes the space of one.

### Why `max-poll-records: 10` and a 15-minute `max.poll.interval.ms` ?

The fetcher makes three requests per record under a 40-per-5-minutes throttle, so one batch can take many minutes. If a consumer doesn't poll within the interval, Kafka assumes it died and reassigns its records, which would fetch events twice and spend the rate limit. Small batches and a long interval prevent that.

### Why three partitions with one consumer each?

Three leaves room to run more consumers in a group later without repartitioning. Today each listener is one thread reading all three. Records with the same key share a partition, so an event's updates stay in order.

### Why JSON and not Avro or Protobuf with a schema registry?

Simplicity at this size: the records are Java records, readable in Kafka UI, and tolerant of added fields ( `ignoreUnknown` ). The trade-off is no enforced schema evolution. With more producers or teams I would add a registry.

## Spring and Java

### What does `@Bean` do, and when do you use it instead of `@Component`?

`@Bean` goes on a method of a `@Configuration` class; Spring calls it once and registers the result. I use it when the object needs construction logic or isn't my class: the `Clock`, the `NewTopic`s, the error handler, the filter registrations, the `TokenService`. `@Component` and its relatives go on my own classes that Spring can construct directly. (Part 3.2)

### Why constructor injection?

Dependencies are explicit and final, the class can't exist half-wired, and tests construct it with fakes without starting Spring.

### Why inject a `Clock`?

So time is an input. Tests check "is this event final 48 hours after it started?" or "does this token expire at 15 minutes?" at exact instants.

### How does `@Transactional` work, and what's the catch?

Spring wraps the bean in a proxy that opens a transaction before the method and commits or rolls back after. The catch: it only applies to calls that come through the proxy, so a method calling another `@Transactional` method on `this` gets no new transaction. Here each transactional method is called from another bean.

### Why `JdbcTemplate` in most places and JPA in one?

JPA came with the first version and suits its small load-modify-save counters. The rest is bulk replaces and reporting joins, which are clearer and faster as SQL. (Part 5.3)

### Why records?

Wire types, responses, settings and query rows are all plain immutable data. A record is that in one line, and Jackson maps it to JSON by field name.

## The sources

### How do you stay under Limitless's rate limit?

`RequestThrottle` keeps a sliding window of the requests sent (40 in 5 minutes, under the 50 allowed) and also reads Limitless's own `ratelimit` header, because the app on the same machine shares the address. Every request passes through it via a `RestClient` interceptor. On a 429 the client honors `Retry-After`.

### How do you decide when an event's standings are final?

By the app's rule: a fetch made 48 hours or more after the event started is final. Until then the event is stored, so it can be seen, and fetched again every 6 hours. The all-time counters wait for the final fetch. (`FetchPolicy`)

### Player-typed names are messy. How do you count them?

`NameStandardizer` maps each item, ability and move to a canonical spelling using the game's own lists: exact match after normalizing, then a single near miss by edit distance, otherwise kept as typed. Species use the app's own vocabulary (`SpeciesVocabulary`), and a golden file checked by both test suites keeps server and app in agreement.

### Why reduce Smogon's files instead of storing them?

A file is up to 7 MB, over the topic's limit and far more than the app shows. The reducer streams it one Pokémon at a time and keeps about 1 MB. Smogon keeps every month, so the raw file can always be fetched again; a version number on the rule makes the server do that when the rule changes. (`LadderReducer`)

## Security

### Why trade a key for a JWT instead of sending the key every time?

The long-lived secret is exposed once every 15 minutes rather than on every request, and a leaked token expires by itself. The server stores only the key's SHA-256 hash.

### What JWT attacks does `verify` guard against?

It pins the algorithm to HS256 before checking the signature (no `alg: none`, no algorithm switching), checks expiry, issuer and audience, and checks that the key the token was issued for is still configured. (Part 6.4)

### Why constant-time comparison?

A normal comparison returns at the first differing byte, so response time leaks how much of a guess was right. `MessageDigest.isEqual` takes the same time regardless, and every configured hash is compared.

### Why not Spring Security?

The need is one check on every request and one token endpoint. A servlet filter and a small service do that in a few hundred lines that can be read end to end. With users, roles or OAuth I would switch.

## Limits, and what I would change

### What are this system's weak points?

- **One of everything.** One broker with replication factor 1, one Postgres, one server instance. A disk loss is recovered from the daily backup and by fetching again, not by failover.
- **Scheduling assumes one instance.** Two instances would both poll. The `event_fetch` table and idempotent stores make that wasteful rather than wrong, but a lock (ShedLock, or a leader) would be the fix. The throttle and the cache are also per instance.
- **Short retries.** Six seconds of retrying doesn't outlast a database restart; those records go to the DLT and need a replay. A longer exponential back-off, or pausing the listener, would be better.
- **Nobody watches the DLTs.** There's no alert on dead letters or on ingest lag yet; it's in the plan's launch phase.
- **The cache is crude.** A map cleared when it passes 500 entries. Fine for one user; a real cache with size and time limits (Caffeine) is the obvious upgrade.
- **ETags save bytes, not work.** The filter hashes a body the server has already produced.
- **Test isolation.** The database slice tests commit rather than roll back, so some tests clear their tables first. Found while writing the ladder tests; noted for a fix.

### How would this scale to many users?

Reads scale first and easily: the API is stateless apart from its cache, so more instances behind a load balancer and a CDN in front (the answers are identical for everyone and already carry caching headers). Writes barely grow with users, since the sources are the same. For more sources or volume: more partitions and consumers per group, a replicated Kafka cluster, and precomputed projections for the heaviest insights.

### What would you build next?

Monitoring (lag, newest event's age, dead letters), a webhook from Limitless in place of polling, and a replay test that rebuilds a table from its topic and checks it is identical.